

TRƯỜNG ĐẠI HỌC SƯ PHẠM HÀ NỘI 2
VIỆN CÔNG NGHỆ THÔNG TIN



HOÀNG ÁNH TUYẾT

**XÂY DỰNG MÔ HÌNH NHẬN DIỆN CỬ CHỈ TAY THỜI
GIAN THỰC BẰNG YOLO VÀ ỨNG DỤNG TRONG ĐIỀU
KHIỂN THAO TÁC**

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC

Chuyên ngành: Công nghệ Thông tin

PHÚ THỌ, 2026

TRƯỜNG ĐẠI HỌC SƯ PHẠM HÀ NỘI 2
VIỆN CÔNG NGHỆ THÔNG TIN



HOÀNG ÁNH TUYẾT

**XÂY DỰNG MÔ HÌNH NHẬN DIỆN CỬ CHỈ TAY THỜI
GIAN THỰC BẰNG YOLO VÀ ỨNG DỤNG TRONG ĐIỀU
KHIỂN THAO TÁC**

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC

Chuyên ngành: Công nghệ Thông tin

Người hướng dẫn khoa học

ThS. Nguyễn Thị Loan

PHÚ THỌ, 2026

LỜI CẢM ƠN

Em xin chân thành gửi lời cảm ơn sâu sắc đến các giảng viên của Viện Công nghệ Thông tin, Trường Đại học Sư phạm Hà Nội 2, đã tận tình giảng dạy, truyền đạt những kiến thức quý báu và tạo điều kiện thuận lợi cho em trong suốt quá trình học tập và hoàn thành khóa luận tốt nghiệp.

Đặc biệt, em xin bày tỏ lòng biết ơn sâu sắc tới **ThS. Nguyễn Thị Loan**, người đã dành nhiều thời gian tận tình hướng dẫn, giải đáp và đưa ra những chỉ dẫn quý báu giúp em có định hướng rõ ràng trong quá trình thực hiện đề tài “Xây dựng mô hình nhận diện cử chỉ tay thời gian thực bằng YOLO và ứng dụng trong điều khiển thao tác”. Sự tận tâm và những kinh nghiệm quý báu của cô đã giúp em từng bước hoàn thiện và phát triển đề tài của mình.

Em cũng xin gửi lời cảm ơn chân thành đến các anh chị và những người có chuyên môn đã sẵn lòng hỗ trợ, chia sẻ kinh nghiệm trong quá trình em thực hiện khóa luận. Đồng thời, em xin trân trọng cảm ơn các bạn đồng khóa, gia đình và bạn bè - những người luôn ở bên động viên, giúp đỡ và tiếp thêm động lực để em hoàn thành đề tài.

Trong quá trình thực hiện, em đã cố gắng hết sức, tập trung và nỗ lực không ngừng để đạt được kết quả tốt nhất. Tuy nhiên, việc mắc phải sai sót là không thể tránh khỏi. Do đó, em mong nhận được sự góp ý, phản hồi từ quý thầy cô và các bạn để đề tài này có thể được phát triển và hoàn thiện hơn.

Em xin trân trọng cảm ơn!

Phú Thọ, tháng 03 năm 2026

Sinh viên thực hiện



Hoàng Ánh Tuyết

LỜI CAM ĐOAN

Em xin cam đoan rằng khóa luận tốt nghiệp với đề tài “Xây dựng mô hình nhận diện cử chỉ tay thời gian thực bằng YOLO và ứng dụng trong điều khiển thao tác” là kết quả nghiên cứu và thực hiện độc lập của cá nhân em dưới sự hướng dẫn tận tình của **ThS. Nguyễn Thị Loan**.

Toàn bộ nội dung, số liệu và kết quả trình bày trong khóa luận được thực hiện trong quá trình nghiên cứu của em và chưa từng được công bố trong bất kỳ công trình nào khác. Các tài liệu, nguồn tham khảo được sử dụng trong khóa luận đều được trích dẫn rõ ràng, đầy đủ và tuân thủ đúng các quy định về đạo đức học thuật.

Em xin hoàn toàn chịu trách nhiệm về tính trung thực, chính xác của các thông tin, dữ liệu và kết quả nghiên cứu được trình bày trong khóa luận này.

Phú Thọ, tháng 03 năm 2026

Sinh viên thực hiện



Hoàng Ánh Tuyết

MỤC LỤC

MỞ ĐẦU	1
1. Lý do chọn đề tài	1
2. Mục đích, nhiệm vụ nghiên cứu.....	2
3. Phương pháp nghiên cứu.....	2
4. Đối tượng và phạm vi nghiên cứu.....	3
5. Cấu trúc khóa luận	3
CHƯƠNG 1: TỔNG QUAN VÀ CƠ SỞ LÝ THUYẾT	4
1.1 Tổng quan về bài toán nhận diện cử chỉ tay	4
1.1.1 Khái niệm và ứng dụng về nhận diện cử chỉ tay.....	4
1.1.2 Các hướng tiếp cận trong nhận diện cử chỉ tay.....	5
1.1.3 Thách thức khi nhận diện cử chỉ tay trong thời gian thực	5
1.2 Tổng quan về học sâu và thị giác máy tính	6
1.2.1 Học sâu và vai trò học sâu trong thị giác máy tính.....	6
1.2.2 Bài toán phát hiện đối tượng và các mô hình phổ biến	7
1.3 Tổng quan về mô hình YOLO	9
1.3.1 Quá trình phát triển và nguyên lý hoạt động của YOLO	9
1.3.2 Lý do lựa chọn YOLOv12	10
1.4 Các chỉ số đánh giá mô hình.....	10
CHƯƠNG 2: PHÂN TÍCH, THIẾT KẾ VÀ XÂY DỰNG MÔ HÌNH	12
2.1 Mô tả bài toán và yêu cầu của hệ thống	12
2.1.1 Mô tả bài toán	12
2.1.2 Yêu cầu chức năng.....	12
2.1.3 Yêu cầu phi chức năng.....	13
2.2 Dữ liệu và quá trình xử lý dữ liệu	14
2.2.1 Nguồn dữ liệu và các lớp cử chỉ tay	14
2.2.2 Quá trình gán nhãn dữ liệu.....	14
2.2.3 Tiền xử lý và phân chia tập dữ liệu.....	15
2.3 Thiết kế hệ thống	18
2.3.1 Kiến trúc tổng thể của hệ thống	18
2.3.2 Luồng xử lý dữ liệu trong hệ thống	18

2.4	Xây dựng và huấn luyện mô hình YOLO	19
2.4.1	Cấu hình của mô hình	19
2.4.2	Thiết lập tham số huấn luyện	20
2.4.3	Quá trình huấn luyện và đánh giá mô hình	21
2.5	Triển khai hệ thống nhận diện thời gian thực	23
2.5.1	Tích hợp mô hình vào hệ thống	23
2.5.2	Xử lý video/camera và hiển thị kết quả	23
CHƯƠNG 3: THỰC NGHIỆM VÀ ĐÁNH GIÁ		25
3.1	Môi trường thực nghiệm	25
3.1.1	Phần cứng và phần mềm sử dụng	25
3.1.2	Thiết lập thực nghiệm	26
3.2	Kịch bản thử nghiệm	27
3.2.1	Mục tiêu thử nghiệm và tiêu chí đánh giá hệ thống	27
3.2.2	Kịch bản và quy trình thực nghiệm	28
3.3	Kết quả huấn luyện	29
3.3.1	Kết quả quá trình huấn luyện lần 1	29
3.3.2	Kết quả quá trình huấn luyện lần 2	37
3.4	Đánh giá mô hình trên tập test	46
3.5	Giao diện hệ thống	47
3.6	Đánh giá hiệu năng thời gian thực và độ bền vững của hệ thống	52
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN		53
TÀI LIỆU THAM KHẢO		55

DANH MỤC CHỮ VIẾT TẮT

Chữ viết tắt	Tên đầy đủ	Giải nghĩa
API	Application Programming Interface	Giao diện lập trình ứng dụng
CIoU	Complete Intersection over Union	Hàm mất mát hộp giới hạn cải tiến
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
COCO	Common Objects in Context	Bộ dữ liệu chuẩn đánh giá phát hiện đối tượng
CPU	Central Processing Unit	Bộ xử lý trung tâm
CVPR	Conference on Computer Vision and Pattern Recognition	Hội nghị thị giác máy tính và nhận dạng mẫu
ELAN	Efficient Layer Aggregation Network	Mạng tổng hợp lớp hiệu quả trong YOLOv12
FPS	Frames Per Second	Số khung hình trên mỗi giây
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
HaGRID	Hand Gesture Recognition Image Dataset	Bộ dữ liệu ảnh nhận diện cử chỉ tay
HGR	Hand Gesture Recognition	Nhận diện cử chỉ tay
HMM	Hidden Markov Model	Mô hình Markov ẩn
HOG	Histogram of Oriented Gradients	Biểu đồ gradient định hướng
HSV	Hue Saturation Value	Không gian màu sắc (Màu sắc, độ bão hòa, độ sáng)

Chữ viết tắt	Tên đầy đủ	Giải nghĩa
IoU	Intersection over Union	Tỷ lệ giao trên hợp (chỉ số đánh giá hộp giới hạn)
mAP	mean Average Precision	Độ chính xác trung bình trên các lớp
R-CNN	Region-based Convolutional Neural Network	Mạng nơ-ron tích chập dựa trên vùng
SIFT	Scale-Invariant Feature Transform	Biến đổi đặc trưng bất biến tỷ lệ
SSD	Single Shot MultiBox Detector	Mô hình phát hiện đối tượng một giai đoạn
YOLO	You Only Look Once	Mô hình phát hiện đối tượng một giai đoạn theo thời gian thực

DANH MỤC BẢNG BIỂU

Bảng 1.1 Các mô hình phổ biến	8
Bảng 2.1 Class_id gán nhãn	15
Bảng 2.2 Cấu hình mô hình	20
Bảng 2.3 Tham số lần huấn luyện 1	20
Bảng 2.4 Tham số lần huấn luyện 2	21
Bảng 3.1 Môi trường huấn luyện	25
Bảng 3.2 Môi trường kiểm thử thực tế	25
Bảng 3.3 Thao tác điều khiển ứng cử chỉ tay	27
Bảng 3.4 So sánh hiệu năng mô hình giữa lần huấn luyện 1 và lần huấn luyện 2	45
Bảng 3.5 Thống kê hiệu năng trong các điều kiện môi trường	52

DANH MỤC HÌNH VẼ

Hình 1.1 Học sâu.....	6
Hình 2.1 Tỷ lệ tập Train/Val/Test lần huấn luyện 1 với 4.800 ảnh (800 ảnh/lớp)	16
Hình 2.2 Tỷ lệ tập Train/Val/Test lần huấn luyện 2 với 6.000 ảnh (1.000 ảnh/lớp) .	17
Hình 2.3 Cấu trúc thư mục YOLO	17
Hình 2.4 Nội dung file gesture.yaml.....	18
Hình 2.5 Tham số cấu hình lần huấn luyện 1	22
Hình 2.6 Tham số cấu hình lần huấn luyện 2.....	22
Hình 2.7 Tích hợp best.pt vào hệ thống nhận diện	23
Hình 2.8 Lệnh khởi tạo webcam	23
Hình 3.1 Biểu đồ quá trình huấn luyện (loss, precision, recall, mAP)	30
Hình 3.2 Đường cong Precision - Recall	30
Hình 3.3 Đường cong F1 - Confidence.....	31
Hình 3.4 Đường cong Precision - Confidence	31
Hình 3.5 Đường cong Recall - Confidence.....	32
Hình 3.6 Confusion matrix của mô hình.....	32
Hình 3.7 Confusion matrix chuẩn hóa	33
Hình 3.8 Phân bố dữ liệu và bounding box trong tập huấn luyện.....	34
Hình 3.9 So sánh ground truth và kết quả dự đoán.....	35
Hình 3.10 Một số batch dữ liệu trong quá trình huấn luyện	37
Hình 3.11 Biểu đồ quá trình huấn luyện (loss, precision, recall, mAP)	38
Hình 3.12 Đường cong Precision - Recall	38
Hình 3.13 Đường cong F1 - Confidence.....	39
Hình 3.14 Đường cong Precision - Confidence	39
Hình 3.15 Đường cong Recall - Confidence.....	40
Hình 3.16 Confusion matrix của mô hình.....	40
Hình 3.17 Confusion matrix chuẩn hóa	41

Hình 3.18 Phân bố dữ liệu và hộp giới hạn trong tập huấn luyện.....	42
Hình 3.19 So sánh ground truth và kết quả dự đoán.....	43
Hình 3.20 Một số batch dữ liệu trong quá trình huấn luyện	44
Hình 3.21 Môi trường mô phỏng 3D trong Unity.....	47
Hình 3.22 Cử chỉ call - Kích hoạt hành động CALL_NURSE.....	48
Hình 3.23 Cử chỉ fist - Kích hoạt hành động CLOSE_DOOR.....	48
Hình 3.24 Cử chỉ palm - Kích hoạt hành động OPEN_DOOR	49
Hình 3.25 Cử chỉ peace - Kích hoạt hành động TOGGLE_BED	49
Hình 3.26 Cử chỉ stop - Kích hoạt hành động TOGGLE_LIGHT.....	50
Hình 3.27 Cử chỉ like - Hành động MOVE_FORWARD.....	50
Hình 3.28 Cử chỉ like - Hành động MOVE_RIGHT	51
Hình 3.29 Cử chỉ like - Hành động MOVE_LEFT.....	51

MỞ ĐẦU

1. Lý do chọn đề tài

Giao tiếp bằng tay là một trong những hành vi tự nhiên nhất của con người, từ hiệu lệnh dừng đến ký hiệu “gọi cho tôi” bằng ngón út và ngón cái. Câu hỏi đặt ra là: Máy tính có thể hiểu được ngôn ngữ tay và có khả năng đủ nhanh để phản hồi ngay lập tức không? Bài toán nhận diện cử chỉ tay thời gian thực (Hand Gesture Recognition - HGR) nổi lên như một giải pháp tiềm năng, cho phép máy tính diễn giải chuyển động bàn tay qua camera thông thường mà không cần thiết bị đeo hay cảm biến chuyên dụng.

Tuy nhiên, thách thức đặt ra yêu cầu mô hình vừa nhận diện chính xác vừa đủ nhanh để phản hồi tức thì. Các phương pháp truyền thống dễ mất ổn định khi điều kiện ánh sáng và nền thay đổi, còn các mô hình học sâu hai giai đoạn mặc dù có độ chính xác cao nhưng lại quá chậm cho tương tác thời gian thực. YOLOv12 là phiên bản được công bố đầu năm 2025, lần đầu tích hợp cơ chế chú ý vào kiến trúc cốt lõi đã cải thiện đáng kể khả năng học đặc trưng nhưng vẫn giữ được tốc độ suy luận phù hợp [1]. Kết hợp với bộ dữ liệu HaGRID quy mô lớn và đa dạng, đây là nền tảng kỹ thuật phù hợp để giải quyết bài toán đặt ra.

Từ những lý do trên, tác giả lựa chọn thực hiện đề tài: **“Xây dựng mô hình nhận diện cử chỉ tay thời gian thực bằng YOLO và ứng dụng trong điều khiển thao tác”**. Thay vì chỉ dừng lại ở trên lý thuyết, đề tài hướng tới việc giải quyết trọn vẹn bài toán từ giai đoạn xử lý dữ liệu đến triển khai ứng dụng thực tế, mang lại những đóng góp thực tiễn: Xây dựng và gán nhãn chuẩn YOLO cho tập dữ liệu gồm 6.000 ảnh được thu thập từ HaGRID, phục vụ trực tiếp cho bài toán nhận diện thời gian thực. Cấu hình và tinh chỉnh mô hình YOLOv12m, ứng dụng cơ chế chú ý để đạt được độ chính xác cao ngay cả trong môi trường phần cứng giới hạn. Hệ thống có thể chuyển thành nhiều lệnh điều hướng khác nhau (trái, phải, tiến), không phải tốn tài nguyên huấn luyện thêm lớp dữ liệu mới. Đặc biệt tích hợp mô hình vào môi trường mô phỏng 3D (Unity) điều khiển các thiết bị sinh hoạt hỗ trợ người bị hạn chế vận động.

2. Mục đích, nhiệm vụ nghiên cứu

Mục đích: Xây dựng mô hình nhận diện thời gian thực ứng dụng YOLOv12 nhằm phát hiện và phân loại cử chỉ tay thông qua dữ liệu hình ảnh từ camera, đồng thời kết hợp ánh xạ cử chỉ tay thành các lệnh điều khiển.

Nhiệm vụ nghiên cứu

Hệ thống hóa lý thuyết về nhận diện cử chỉ tay, tương tác người - máy không tiếp xúc, mạng nơ-ron tích chập và kiến trúc YOLO.

Xây dựng và huấn luyện mô hình YOLOv12 trên bộ dữ liệu HaGRID, tập trung nhận diện 6 lớp cử chỉ tay (call, fist, like, palm, peace, stop) với độ chính xác cao, sử dụng tài nguyên tính toán phổ thông (Google Colab GPU Tesla T4).

Đánh giá định lượng hiệu suất mô hình thông qua các chỉ số Precision, Recall, F1-score, mAP@50 và mAP@50-95, kết hợp phân tích ma trận nhầm lẫn cho từng lớp cử chỉ.

Triển khai ứng dụng nhận diện cử chỉ tay thời gian thực bằng Python, kết hợp PyQt6 và OpenCV để xử lý hình ảnh từ webcam; ánh xạ các cử chỉ đã nhận diện thành các hành động điều khiển cụ thể (call nurse, close door, move forward, move right, move left, open door, toggle light, toggle bed) trong môi trường 3D được xây dựng trên Unity.

Đánh giá hiệu suất thực tế của hệ thống qua FPS, độ ổn định khi nhận diện và tính chính xác khi thực hiện các lệnh điều khiển bằng cử chỉ tay.

3. Phương pháp nghiên cứu

Phương pháp nghiên cứu tài liệu: Thu thập và tổng hợp tài liệu, các công trình nghiên cứu về nhận diện cử chỉ tay và các hệ thống nhận diện cử chỉ tay đã có. Qua đó, xác định yếu tố quan trọng trong việc xây dựng mô hình nhận diện hiệu quả, phù hợp thời gian thực.

Phương pháp thu thập dữ liệu: Tiến hành thu thập dữ liệu ảnh từ bộ dữ liệu HaGRID, lựa chọn lớp cử chỉ phù hợp với mục tiêu nghiên cứu. Đồng thời, chuẩn hóa và gán nhãn dữ liệu để đảm bảo chất lượng đầu vào cho quá trình huấn luyện.

Phương pháp thực nghiệm: Tiến hành huấn luyện mô hình trên bộ dữ liệu đã chuẩn hóa, theo dõi quá trình học và điều chỉnh các tham số nâng cao độ chính xác của mô hình. Quá trình thực nghiệm giúp đánh giá khả năng nhận diện của hệ thống trong các điều kiện khác nhau.

Phương pháp phân tích và thiết kế hệ thống: Phân tích yêu cầu của bài toán nhận diện cử chỉ tay trong thời gian thực, từ đó xây dựng cấu trúc hệ thống, thiết kế mô hình và quy trình xử lý dữ liệu.

Phương pháp kiểm thử và đánh giá: Áp dụng các phương pháp kiểm thử để đánh giá hiệu năng của mô hình qua các chỉ số như Precision, Recall, F1-score và mAP. Đồng thời, tiến hành thử nghiệm hệ thống trên webcam thực tế nhằm đánh giá tốc độ xử lý và độ ổn định, từ đó đề xuất các hướng cải tiến.

4. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài là mô hình nhận diện cử chỉ tay ứng dụng YOLOv12. Trong quá trình thực hiện, đề tài tập trung vào việc xây dựng một mô hình có khả năng nhận diện 6 cử chỉ tay cơ bản gồm call, fist, like, palm, peace và stop. Bên cạnh đó, đề tài sử dụng ngôn ngữ Python cùng với các thư viện như Ultralytics và OpenCV để triển khai phần nhận diện, đồng thời kết hợp với Unity nhằm xây dựng môi trường mô phỏng và thực hiện điều khiển các hành động tương ứng với từng cử chỉ tay.

Phạm vi nghiên cứu của đề tài tập trung vào việc nhận diện các cử chỉ tay tĩnh thời gian thực thông qua webcam. Mô hình được huấn luyện và đánh giá trên bộ dữ liệu HaGRID với 6 lớp cử chỉ đã lựa chọn. Sau khi huấn luyện, hệ thống được triển khai để thử nghiệm trong môi trường thực tế, với điều kiện thay đổi ánh sáng, góc nhìn và khoảng cách. Các cử chỉ nhận diện sẽ được ánh xạ thành các hành động cụ thể trong môi trường mô phỏng nhằm kiểm tra khả năng ứng dụng của hệ thống.

5. Cấu trúc khóa luận

Ngoài phần mở đầu, kết luận, hướng phát triển, và tài liệu tham khảo, khóa luận được tổ chức thành 3 chương:

Chương 1: Tổng quan và cơ sở lý thuyết.

Chương 2: Phân tích, thiết kế và xây dựng mô hình.

Chương 3: Thử nghiệm và đánh giá.

CHƯƠNG 1: TỔNG QUAN VÀ CƠ SỞ LÝ THUYẾT

Chương 1 trình bày tổng quan về bài toán nhận diện cử chỉ tay và các hướng tiếp cận phổ biến hiện nay. Đồng thời, cung cấp cơ sở lý thuyết về học sâu, mạng nơ-ron tích chập, và đi sâu vào kiến trúc cũng như nguyên lý hoạt động của mô hình phát hiện đối tượng YOLO. Cuối cùng, các chỉ số đánh giá hiệu năng mô hình cơ bản cũng được giới thiệu nhằm làm cơ sở nền tảng cho các bước thực nghiệm ở những chương tiếp theo.

1.1 Tổng quan về bài toán nhận diện cử chỉ tay

1.1.1 Khái niệm và ứng dụng về nhận diện cử chỉ tay

Cử chỉ tay là một trong những phương thức giao tiếp phi ngôn ngữ lâu đời và tự nhiên nhất của con người. Trong tương tác hằng ngày, con người sử dụng cử chỉ tay để truyền đạt ý định, cảm xúc và mệnh lệnh một cách bản năng, thường song song hoặc thay thế cho lời nói. Cử chỉ tay được chia thành hai nhóm chính: Cử chỉ tĩnh là bàn tay giữ nguyên một tư thế cố định trong suốt quá trình thực hiện, không có chuyển động. Cử chỉ động: Bàn tay di chuyển theo một quỹ đạo nhất định trong không gian và thời gian[2].

Nhận diện cử chỉ tay (Hand Gesture Recognition - HGR) là quá trình sử dụng các kỹ thuật xử lý ảnh, thị giác máy tính và học máy để phát hiện và phân loại các cử chỉ của bàn tay con người từ hình ảnh hoặc video. Hệ thống phân tích hình dạng, chuyển động của bàn tay, chuyển đổi thành thông tin hoặc lệnh điều khiển trong các ứng dụng tương tác người - máy.

Về ứng dụng, bài toán nhận diện cử chỉ tay đã và đang được triển khai thực tế ở nhiều lĩnh vực:

Trong lĩnh vực công nghiệp ô tô thông minh: Hệ thống nhận diện cử chỉ tay giúp tài xế không phải rời mắt khỏi đường đi, giải quyết bài toán an toàn giao thông. Hệ thống Depthsense của Sony trên các dòng xe BMW giúp người lái chỉ cần xoay ngón tay để chỉnh âm lượng hoặc vẫy tay để từ chối cuộc gọi. Hệ thống MBUX Interior Assistant trên Mercedes-Benz S-Class giúp tài xế nhận diện hướng vẫy tay của hành khách và tự động điều chỉnh đèn nội thất.

Trong nền tảng họp trực tuyến như Zoom và Microsoft Teams (2022) cho phép người dùng giơ lòng bàn tay hoặc ngón tay cái trước camera để kích hoạt các biểu tượng cảm xúc.

Trong y tế và phục hồi chức năng: Hệ thống Gesture4D giúp bác sĩ thực hiện các thao tác phóng to, thu nhỏ hoặc xoay ảnh chụp X-quang và MRI trên màn hình hoàn toàn không chạm. Đối với những người khuyết tật vận động, nhận diện cử chỉ tay giúp tương tác với các thiết bị sinh hoạt, không cần sức lực vật lý.

Trong các dự án nhân đạo cho người khiếm thính do Google SignAll, MotionSavvy phát minh hỗ trợ chuyển ngôn ngữ ký hiệu sang văn bản và giọng nói theo thời gian thực, xóa bỏ rào cản ngôn ngữ cho cộng đồng 430 triệu người người khuyết tật trên toàn thế giới [3].

1.1.2 Các hướng tiếp cận trong nhận diện cử chỉ tay

Có ba hướng tiếp cận chính trong bài toán nhận diện cử chỉ tay:

Cảm biến phần cứng: Sử dụng thiết bị như Leap Motion Controller, Microsoft Kinect và Data Glove thu thập dữ liệu 3D về vị trí các khớp ngón tay. Ưu điểm là độ chính xác về vị trí các khớp ngón tay rất cao nhưng chi phí thiết bị đắt, người dùng phải đứng trong vùng hoạt động hạn chế và rất khó triển khai rộng rãi.

Xử lý ảnh truyền thống: Dựa trên các kỹ thuật như phân đoạn màu da trong không gian HSV, trích xuất đặc trưng HOG (Histogram of Oriented Gradients), SIFT (Scale-Invariant Feature Transform), kết hợp với mô hình xác suất ẩn HMM để phân loại. Nhóm phương pháp này xử lý nhanh, đơn giản, độ chính xác cao, đạt 80 - 90% trong điều kiện kiểm nhưng nhạy cảm thay đổi ánh sáng và màu da đa dạng, hiệu suất giảm đáng kể trong môi trường thực tế.

Học sâu: Các mạng nơ-ron tích chập tự học đặc trưng từ dữ liệu, không cần thiết kế thủ công, tổng quát hóa trên dữ liệu đa dạng. Phương pháp này có khả năng cân bằng tốt giữa tốc độ và độ chính xác.

1.1.3 Thách thức khi nhận diện cử chỉ tay trong thời gian thực

Sự thay đổi điều kiện ánh sáng: Độ sáng, hướng chiếu sáng và bóng đổ thay đổi liên tục theo điều kiện môi trường. Molchanov và các cộng sự (2015) [4] đã ghi nhận rõ trong bối cảnh ô tô: “Ánh sáng bên ngoài chiếu qua kính xe thay đổi theo tốc độ di chuyển và góc mặt trời khiến các phương pháp dựa trên màu sắc da tay hoàn toàn thất bại, buộc nhóm tác giả phải chuyển sang dữ liệu hồng ngoại và độ sâu để loại bỏ sự phụ thuộc vào ánh sáng thông thường.”

Nền phức tạp: Trong môi trường thực tế, màu sắc của bàn tay có thể gần giống với màu nền hoặc các vật thể khác trong ảnh, thuật toán sẽ khó phân biệt bàn tay với môi trường xung quanh.

Sự đa dạng bàn tay người: Kích thước, hình dạng, màu da và cách thực hiện cử chỉ khác nhau giữa các cá nhân theo giới tính, độ tuổi và chủng tộc.

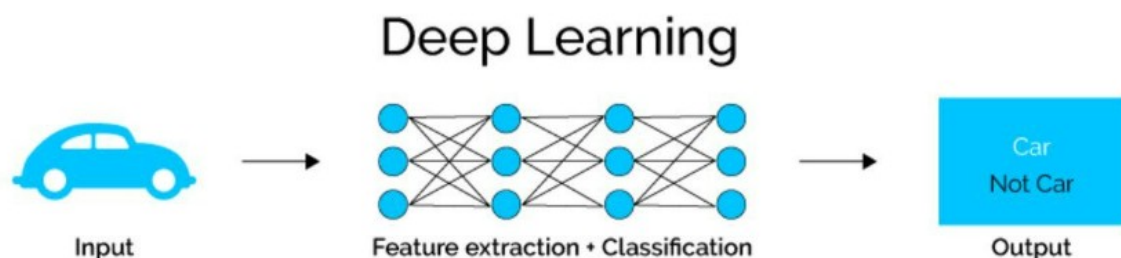
Góc nhìn và khoảng cách: Cùng một cử chỉ có thể xuất hiện dưới nhiều góc độ và ở khoảng cách khác nhau với camera. HaGRID thu thập ảnh ở khoảng cách 0,5 đến 4 mét để bao phủ sự biến thiên thực tế này.

Yêu cầu thời gian thực: Trong ứng dụng điều khiển tương tác, độ trễ phải dưới 100ms để đảm bảo trải nghiệm mượt mà. Hệ thống cần đạt ít nhất 25 - 30 FPS trở lên.

1.2 Tổng quan về học sâu và thị giác máy tính

1.2.1 Học sâu và vai trò học sâu trong thị giác máy tính

Học sâu (Deep Learning) sử dụng mạng nơ-ron nhiều tầng tự động học biểu diễn đặc trưng từ dữ liệu thô. Mô hình tự học toàn bộ quá trình qua huấn luyện: từ đặc trưng đơn giản (cạnh, góc) ở lớp nông đến đặc trưng trừu tượng (hình dạng, kết cấu) ở lớp sâu hơn. Lý thuyết về mạng nơ-ron xuất hiện từ năm 1940 - 1980, và đến năm 2012 mới thực sự bùng nổ. Bước ngoặt lớn đến từ sự hội tụ của ba yếu tố: lượng dữ liệu khổng lồ (đặc biệt là bộ ImageNet với hơn 1,2 triệu ảnh được gán nhãn), sức mạnh tính toán từ GPU và mô hình AlexNet của Krizhevsky cùng cộng sự [5]. Tại cuộc thi ILSVRC (2012), AlexNet gây chấn động khi giảm tỷ lệ lỗi phân loại ảnh từ 26% xuống chỉ còn 15,3% [5] - một bước nhảy vọt chưa từng có. Từ đó, học sâu liên tục phá kỷ lục trong hàng loạt bài toán từ phân loại ảnh, phát hiện đối tượng đến xử lý ngôn ngữ tự nhiên và nhiều lĩnh vực khác.



Hình 1.1 Học sâu

Học sâu đã thay đổi hoàn toàn cục diện của thị giác máy tính. Vai trò cốt lõi nhất nằm ở khả năng học tự động từ đầu đến cuối. Nếu như các phương pháp xử lý ảnh truyền thống đòi hỏi kỹ sư phải tính toán và thiết kế các bộ trích xuất đặc trưng thủ công, vừa tốn thời gian vừa thiếu linh hoạt, thì các mạng nơ-ron học sâu lại tự động trích xuất và biểu diễn dữ liệu. Mô hình có thể tự nhận diện từ đặc trưng cơ bản nhất cho đến các chi tiết ngữ nghĩa phức tạp.

Cơ chế tự học, kết hợp với lượng dữ liệu khổng lồ và sức mạnh tính toán của GPU, học sâu đã khắc phục được điểm yếu chí mạng của phương pháp cũ. Các hệ thống hiện đại có khả năng thích nghi cực tốt với môi trường thực tế, vượt qua các rào cản về sự thay đổi ánh sáng, nền nhiễu, hay góc nhìn đa dạng. Sự phát triển vượt bậc này không chỉ dừng lại ở phân loại ảnh đơn thuần mà còn tạo nền tảng vững chắc cho các bài toán phức tạp hơn như phát hiện đối tượng thời gian thực, đáp ứng hoàn hảo cho các ứng dụng tương tác người - máy.

Mạng nơ-ron tích chập có ba đặc điểm kiến trúc cốt lõi: Kết nối cục bộ, chia sẻ trọng số và tính bất biến dịch chuyển. Kiến trúc này tối ưu hóa số lượng tham số mà vẫn bảo toàn thông tin không gian của ảnh. Từ LeNet-5, một trong những mô hình tiên phong ra đời năm 1998, đến bước ngoặt AlexNet năm 2012, rồi sau đó là ResNet năm 2015 với cơ chế kết nối dư giúp giải quyết triệt để vấn đề suy giảm đạo hàm khi mạng quá sâu, có thể thấy mạng nơ-ron tích chập đã dần trở thành bộ khung chuẩn cho hầu hết các bài toán xử lý hình ảnh phức tạp hiện nay.

1.2.2 Bài toán phát hiện đối tượng và các mô hình phổ biến

Phát hiện đối tượng (Object Detection) là bài toán hệ thống cần xác định vị trí và loại của các đối tượng xuất hiện trong ảnh hoặc video. Kết quả được biểu diễn bằng các hộp giới hạn kèm theo nhãn lớp và xác suất dự đoán. Các mô hình phát hiện đối tượng chia thành hai nhóm kiến trúc chính:

Mô hình hai giai đoạn như R-CNN, Fast R-CNN và Faster R-CNN xử lý theo hai bước: Một mạng đề xuất vùng tạo ra các vùng ứng viên có thể chứa đối tượng, sau đó một mạng phân loại xử lý từng vùng đó. Độ chính xác rất cao, nhưng Faster R-CNN là phiên bản nhanh nhất thường chỉ đạt 10 - 20 FPS trên GPU thông thường, không đủ cho ứng dụng thời gian thực.

Mô hình một giai đoạn như YOLO, SSD và RetinaNet phát hiện và phân loại đối tượng trong một lần xử lý duy nhất, không có bước đề xuất vùng. Nhược điểm

ban đầu kém chính xác hơn với các vật thể nhỏ. Các phiên bản YOLO gần đây đã rút ngắn đáng kể khoảng cách về độ chính xác trong khi vẫn duy trì tốc độ.

Học sâu phát triển kéo theo sự ra đời của hàng loạt kiến trúc mạng nơ-ron, trở thành nền tảng quan trọng cho thị giác máy tính. Ban đầu, các nhà nghiên cứu tập trung xây dựng backbone mạnh như VGGNet để trích xuất đặc trưng hình ảnh hiệu quả. Sau đó, họ dần bổ sung phần neck và head để tạo nên những mô hình phát hiện đối tượng hoàn chỉnh. Hiện nay, các mô hình phát hiện đối tượng được chia thành hai hướng chính: hai giai đoạn và một giai đoạn. Các mô hình hai giai đoạn như Faster R-CNN ưu tiên độ chính xác cao nhờ cơ chế đề xuất vùng ứng viên, nhưng tốc độ xử lý chậm. Ngược lại, các mô hình một giai đoạn như SSD và đặc biệt là YOLO cho phép dự đoán trực tiếp tọa độ hộp giới hạn cùng nhãn lớp chỉ trong một lần quét, phù hợp thời gian thực.

Bảng 1.1 Các mô hình phổ biến

Mô hình	Năm	Loại	Ưu điểm	Nhược điểm
VGGNet	2014	Backbone	- Kiến trúc đơn giản, nhất quán - Dễ huấn luyện lại	Tiêu thụ bộ nhớ lớn (138 triệu tham số)
Faster R-CNN	2015	Hai giai đoạn	Độ chính xác rất cao	- Tốc độ xử lý chậm - Không đáp ứng thời gian thực
SSD	2016	Một giai đoạn	- Nhanh - Xử lý đa tỷ lệ tốt	- Xử lý phức tạp - Khó tùy chỉnh
YOLO	2016 - Nay	Một giai đoạn	- Rất nhanh (45-155 FPS) - Cân bằng tốc độ và độ chính xác - Phù hợp thời gian thực	- Cần bộ dữ liệu lớn - Hiệu suất tụt giảm khi nền phức tạp hoặc nhiều nhiễu

1.3 Tổng quan về mô hình YOLO

1.3.1 Quá trình phát triển và nguyên lý hoạt động của YOLO

YOLO (You Only Look Once) được Joseph Redmon, Santosh Divvala, Ross Girshick và Ali Farhadi công bố tại CVPR 2016, đã tái định nghĩa bài toán nhận diện đối tượng thời gian thực [8]. Ý tưởng chính của YOLO là coi bài toán như một bài toán hồi quy, trong đó mạng nơ-ron dự đoán trực tiếp vị trí hộp giới hạn và xác suất lớp từ toàn bộ ảnh đầu vào.

YOLOv1 chia ảnh đầu vào thành lưới $S \times S$ ô (mặc định 7×7). Mỗi ô phát hiện các đối tượng có tâm rơi vào ô đó, dự đoán B hộp giới hạn cùng độ tin cậy và C xác suất lớp - tất cả được tính toán song song trong một lần forward pass [8]. YOLOv1 đạt 45 FPS trên GPU Titan X, nhanh hơn khoảng 100 lần so với R-CNN và nhanh hơn gần 10 lần so với Faster R-CNN tại thời điểm đó. Phiên bản rút gọn Fast YOLO thậm chí đạt 155 FPS. Tuy nhiên có một số hạn chế rõ ràng: Mỗi ô chỉ dự đoán được một lớp đối tượng và khó xử lý vật thể nhỏ và các đối tượng gần nhau. YOLO đã phát triển qua bốn giai đoạn chính:

Giai đoạn nền tảng YOLOv2 và YOLOv3 (2017 - 2018): YOLOv2 bổ sung kỹ thuật chuẩn hóa theo lô giúp mô hình hội tụ nhanh và ổn định hơn. Đặc biệt, thay vì thiết lập kích thước thủ công, các hộp neo trong YOLOv2 được khởi tạo tự động dựa trên thuật toán phân cụm từ chính tập dữ liệu huấn luyện, giúp mô hình phát hiện chính xác nhiều lớp đối tượng hơn.

Giai đoạn tối ưu hóa YOLOv4 và YOLOv5 (2020): YOLOv4 tích hợp kỹ thuật tăng cường dữ liệu dạng khảm, hàm mất mát CIOU và mạng xương sống CSPDarknet, đẩy mAP trên COCO lên 43,5% ở 65 FPS. YOLOv5 chuyển sang PyTorch, dễ triển khai rộng rãi.

Giai đoạn loại bỏ hộp neo YOLOv6, YOLOv7, YOLOv8, YOLOv11 (2022 - 2024): Chuyển hoàn toàn sang kiến trúc không dùng hộp neo, tách biệt nhánh phân loại và hồi quy vị trí, mạng xương sống liên tục được tối ưu hóa nhằm giảm tham số, không mất độ chính xác.

Giai đoạn tập trung vào cơ chế chú ý YOLOv12 (2025): Cơ chế chú ý lần đầu được tích hợp, thay thế vai trò trung tâm của mạng nơ-ron tích chập.

1.3.2 Lý do lựa chọn YOLOv12

Được công bố đầu năm 2025, YOLOv12 đánh dấu bước chuyển mình lịch sử: Tích hợp cơ chế chú ý cải thiện khả năng học đặc trưng và phát hiện đối tượng trong các cảnh phức tạp [1]. Theo nghiên cứu của Tian, Ye và Doermann (2025), YOLOv12 tích hợp ba công nghệ lõi: Cơ chế chú ý vùng (Area Attention - A^2): Chia nhỏ bản đồ đặc trưng thành các vùng cục bộ giảm chi phí tính toán của self-attention khi xử lý các bản đồ đặc trưng lớn. Mạng tổng hợp R-ELAN: Bổ sung kết nối dư vào cấu trúc tổng hợp đặc trưng. Điều này sẽ giúp giảm tình trạng nghẽn đạo hàm, hội tụ ổn định khi tăng quy mô. FlashAttention: Tối ưu hóa thuật toán I/O trên bộ nhớ GPU. Đồng thời, triệt tiêu nút thắt truy cập dữ liệu, giúp tăng tốc quá trình suy luận.

YOLOv12 có khả năng cân bằng tốt tốc độ xử lý và độ chính xác, tuy nhiên, cần tập dữ liệu lớn, tài nguyên tính toán và hiệu suất có thể giảm nếu dữ liệu không đa dạng hoặc chứa nhiều nhiễu. Đề tài lựa chọn sử dụng phiên bản YOLOv12m (Medium). Mặc dù có kích thước lớn hơn phiên bản Nano (n), YOLOv12m cung cấp sự cân bằng tối ưu giữa độ chính xác và tốc độ xử lý, ứng dụng tốt trong thời gian thực. Trên các tài nguyên phần cứng như GPU Tesla T4 (cho môi trường huấn luyện) và CPU Intel Core i5 (cho môi trường kiểm thử thực tế), YOLOv12m vẫn đảm bảo tốc độ phản hồi thời gian thực (trên 25 FPS) mà không làm suy giảm hiệu năng nhận diện đối với bài toán 6 lớp cử chỉ tay.

1.4 Các chỉ số đánh giá mô hình

Precision và Recall

Đánh giá chất lượng mô hình nhận diện đối tượng, có bốn trường hợp:

- Dương tính đúng (TP): Mô hình nhận diện đúng đối tượng, hộp giới hạn dự đoán khớp khá tốt với nhãn thực tế.
- Dương tính giả (FP): Mô hình báo có đối tượng nhưng thực tế không có.
- Âm tính giả (FN): Đối tượng thực tế có trong ảnh nhưng mô hình bỏ sót, không phát hiện ra.
- Âm tính đúng (TN): Vùng nền không có đối tượng và mô hình không phát hiện.

Precision - Độ tin cậy: Tỷ lệ số mẫu dự đoán đúng trong tất cả các mẫu mô hình dự đoán dương tính [6]. Precision càng cao, mô hình ít báo nhầm.

$$Precision = \frac{TP}{TP+FP}$$

Recall - Độ phủ của model: Tỷ lệ dự đoán đúng trên toàn bộ mẫu dữ liệu dương tính thực tế [6]. Recall cao cho thấy mô hình ít bỏ sót đối tượng.

$$Recall = \frac{TP}{TP + FN}$$

Khi tăng độ tin cậy, Precision tăng lên vì mô hình chỉ giữ lại dự đoán khá chắc chắn, Recall giảm vì dễ bỏ sót một số đối tượng. Ngược lại, khi giảm độ tin cậy, Recall tăng nhưng Precision giảm.

F1-score

F1-score là điểm hòa giữa Precision và Recall [6]:

$$F1\ Score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

F1-score cao cho thấy mô hình hoạt động tốt ở cả hai phía Precision và Recall, không bị lệch về một phía. Trong đề tài, F1-score được tính riêng cho từng lớp cử chỉ để xác định mô hình xử lý tốt hay chưa tốt.

mAP

mAP - Độ chính xác trung bình: Mức độ chính xác tổng hợp của mô hình khi dự đoán cả lớp đối tượng và vị trí bounding box. Giá trị mAP dao động từ 0 đến 1. Khi giá trị càng cao thì mô hình càng hiệu quả.

AP (Average Precision): Đo lường độ chính xác của mô hình cho một lớp cụ thể, dựa trên đường cong Precision - Recall.

mAP@50: tính AP với ngưỡng IoU = 0,5 - để đạt điểm cao hơn, so sánh nhanh giữa các mô hình.

mAP@50 - 95: Trung bình trên 10 ngưỡng IoU từ 0,5 đến 0,95 – chặt chẽ, yêu cầu bounding box phải khớp chính xác với thực tế.

CHƯƠNG 2: PHÂN TÍCH, THIẾT KẾ VÀ XÂY DỰNG MÔ HÌNH

Chương 2 thực hiện quá trình phân tích, thiết kế hệ thống nhận diện cử chỉ tay. Nội dung chương bắt đầu từ việc mô tả bài toán, xác định yêu cầu hệ thống, cho đến quy trình thu thập và chuẩn hóa bộ dữ liệu. Trọng tâm của chương nằm ở việc thiết lập cấu hình, tinh chỉnh siêu tham số và triển khai huấn luyện YOLOv12 qua hai giai đoạn. Cuối cùng, chương trình bày giải pháp tích hợp mô hình học sâu vào ứng dụng xử lý luồng video thời gian thực.

2.1 Mô tả bài toán và yêu cầu của hệ thống

2.1.1 Mô tả bài toán

Bài toán nghiên cứu được chia thành hai giai đoạn chính:

Giai đoạn 1: Xây dựng hệ thống nhận diện cử chỉ tay

Hệ thống nhận đầu vào là luồng video thời gian thực từ webcam thông thường. Mỗi khung hình sẽ được tự động điều chỉnh kích thước về 640×640 pixel trước khi đưa vào dự đoán. Hệ thống phải phát hiện vị trí bàn tay, phân loại chính xác 6 cử chỉ và hiển thị ngay kết quả lên màn hình (bao gồm hộp giới hạn, tên cử chỉ, độ tin cậy và FPS). Bài toán yêu cầu tốc độ phản hồi nhanh, độ trễ không được vượt quá 200ms.

Giai đoạn 2: Ứng dụng mô hình vào điều khiển thiết bị

Kết quả nhận diện từ mô hình được sử dụng để xây dựng các thao tác tương tác không chạm, hỗ trợ người dùng cần hỗ trợ vận động trong việc điều khiển các thiết bị sinh hoạt. Trong phạm vi đề tài, các cử chỉ được ánh xạ thành lệnh điều khiển trong môi trường mô phỏng 3D.

2.1.2 Yêu cầu chức năng

Yêu cầu chức năng của hệ thống được xác định trên hai thành phần chính.

Đối với mô hình nhận diện cử chỉ tay:

Ảnh đầu vào kích thước 640×640 pixel và xuất ra danh sách các hộp giới hạn kèm nhãn lớp cử chỉ và độ tin cậy tương ứng cho mỗi bàn tay được phát hiện trong ảnh.

Mô hình phải nhận diện chính xác 6 lớp cử chỉ tay tĩnh: call, fist, like, palm, peace và stop.

Mô hình phải xử lý được trường hợp ảnh có nhiều bàn tay cùng lúc, bàn tay ở kích thước và khoảng cách khác nhau, nền phức tạp.

Kết quả dự đoán phải kèm độ tin cậy. Hệ thống chỉ hiển thị kết quả khi độ tin cậy vượt ngưỡng được cấu hình, lọc bỏ các dự đoán không chắc chắn.

Đối với ứng dụng thời gian thực:

Ứng dụng phải đọc luồng video từ webcam, xử lý từng khung hình qua mô hình đã huấn luyện và hiển thị kết quả nhận diện trực tiếp lên khung hình với hộp giới hạn, nhãn lớp và kèm độ tin cậy.

Cho phép người dùng khởi động, dừng nhận diện và thoát chương trình bằng phím tắt đơn giản.

FPS phải hiển thị liên tục trên màn hình để người dùng theo dõi hiệu suất.

Ứng dụng được hoạt động được trên Windows 10/11 với Python 3.10 trở lên, không yêu cầu cài đặt phần mềm ngoài các thư viện Python thông dụng.

2.1.3 Yêu cầu phi chức năng

Yêu cầu phi chức năng xác định các ràng buộc về chất lượng mà hệ thống phải đạt được trong quá trình vận hành.

Về hiệu suất:

Mô hình phải đạt tối thiểu 25 FPS khi chạy trên GPU. Trên CPU, ứng dụng cần đạt ít nhất 10 FPS;

Độ chính xác mAP@50 trên tập kiểm thử phải đạt tối thiểu 80%;

Độ trễ phản hồi từ khi thực hiện cử chỉ đến khi hiển thị phải dưới 200ms trong điều kiện có GPU.

Về độ tin cậy:

Hoạt động ổn định liên tục ít nhất 30 phút, không bị treo hoặc crash trong điều kiện ánh sáng và nền thông thường;

Độ chính xác phải được duy trì tương đối ổn định khi điều kiện ánh sáng thay đổi trong phạm vi bình thường.

Về bảo trì và mở rộng:

Mã nguồn phải được tổ chức rõ ràng theo module, tách biệt phần inference, phần xử lý video và phần hiển thị để bảo trì và nâng cấp;

Dễ dàng mở rộng sang lớp cử chỉ mới bằng cách thêm dữ liệu và huấn luyện lại mô hình, không cần thay đổi kiến trúc tổng thể.

2.2 Dữ liệu và quá trình xử lý dữ liệu

2.2.1 Nguồn dữ liệu và các lớp cử chỉ tay

Bộ dữ liệu sử dụng trong đề tài là HaGRID (Hand Gesture Recognition Image Dataset) phiên bản 1.0, được công bố tại hội nghị WACV 2024 bởi Kapitanov và các cộng sự. [7]. HaGRID chứa 554.800 ảnh FullHD (1920×1080) phân thành 18 lớp cử chỉ, được thu thập từ 37.583 người dùng [7]. Điểm nổi bật của HaGRID nhờ tính đa dạng cao về chủng tộc, giới tính, độ tuổi, màu da, khoảng cách (0,5 - 4 mét), điều kiện ánh sáng và nền ảnh thực tế. So sánh với các bộ dữ liệu nhận diện cử chỉ tay khác, HaGRID vượt trội rõ rệt về quy mô và độ đa dạng. Hầu hết các dataset cũ (NUS Hand Posture, OUHANDS, Senz3D) chỉ có vài nghìn ảnh và thu thập trong điều kiện kiểm soát chặt chẽ, dẫn đến mô hình huấn luyện trên đó khi đem ra dùng thực tế thường sụt giảm hiệu suất đáng kể.

Bộ dữ liệu quy mô lớn này là yếu tố cốt lõi cho quá trình huấn luyện mô hình YOLOv12. Sự phong phú về bối cảnh không gian và đặc điểm con người trong HaGRID giúp mô hình học được các đặc trưng mang tính tổng quát cao, hạn chế tối đa hiện tượng quá khớp trên một tập mẫu hẹp. Điều này đặc biệt quan trọng đảm bảo độ khả thi của đề tài khi triển khai ứng dụng tương tác hỗ trợ người khuyết tật, nơi môi trường sinh hoạt, điều kiện ánh sáng và thiết bị camera của mỗi người hoàn toàn khác biệt.

Đề tài chọn lọc 6 lớp phổ biến, hình dạng trực quan và tính ứng dụng cao trong tương tác người - máy. Cụ thể:

-  Call: Tay làm dấu điện thoại (ngón cái và ngón út duỗi, ba ngón còn lại gập).
-  Fist: Nắm tay hoàn toàn.
-  Like: Ngón cái giờ lên.
-  Palm: Lòng bàn tay xòe hoàn toàn.
-  Peace: Dấu chữ V (ngón trỏ và ngón giữa duỗi).
-  Stop: Bàn tay giờ ra, ngón cái gập vào lòng bàn tay.

2.2.2 Quá trình gán nhãn dữ liệu

Ảnh được gán nhãn theo định dạng YOLO, mỗi ảnh đi kèm một file .txt cùng tên. Cú pháp với tọa độ chuẩn hóa về [0, 1] so với kích thước ảnh [8]:

`class_id center_x center_y width height`

Công cụ sử dụng để gán nhãn là LabelImg.

Quy trình gán nhãn gồm 4 bước:

Bước 1: Quá trình thu thập dữ liệu chia làm hai giai đoạn. Lần đầu, đề tài thu thập 800 ảnh cho mỗi lớp (tổng 4.800 ảnh). Sau đó, ở giai đoạn tinh chỉnh, dữ liệu được bổ sung thêm để đạt mức 1.000 ảnh cho mỗi lớp (tổng cộng 6.000 ảnh). Ảnh được sắp xếp vào các thư mục riêng biệt: call, fist, palm, peace, stop, like.

Bước 2: Cấu hình LabelImg định dạng YOLO: Chuyển sang YOLO mode: View → Auto Save Mode - Format → YOLO. Sau đó, trỏ đường dẫn đến thư mục ảnh và thư mục lưu nhãn tương ứng.

Bước 3: Vẽ hộp giới hạn bao quanh vùng bàn tay và chọn nhãn lớp tương ứng. Mỗi ảnh trong bộ dữ liệu chứa một bàn tay chính, do đó mỗi file nhãn chỉ có một dòng.

Bước 4: Ánh xạ `class_id` theo thứ tự từ 0 đến 5:

Bảng 2.1 Class_id gán nhãn

<code>class_id</code>	Nhãn tương ứng
0	call
1	fist
2	palm
3	peace
4	stop
5	like

Thứ tự này được khai báo thống nhất trong file `gesture.yaml`.

Kết quả: 6.000 file nhãn tương ứng 6.000 ảnh, không có ảnh nào bị bỏ sót. Toàn bộ nhãn được kiểm tra thủ công, loại bỏ hộp giới hạn sai vị trí hoặc set nhầm lớp.

2.2.3 Tiền xử lý và phân chia tập dữ liệu

Tiền xử lý ảnh: Dữ liệu ảnh được chuẩn hóa đồng bộ về kích thước 640×640 pixel trước khi huấn luyện. Quá trình này sử dụng kỹ thuật đệm viền để giữ nguyên

tỷ lệ khung hình, tránh méo ảnh. Việc chuẩn hóa màu sắc được thực hiện tự động thông qua quy trình xử lý của thư viện Ultralytics mà không cần can thiệp thủ công.

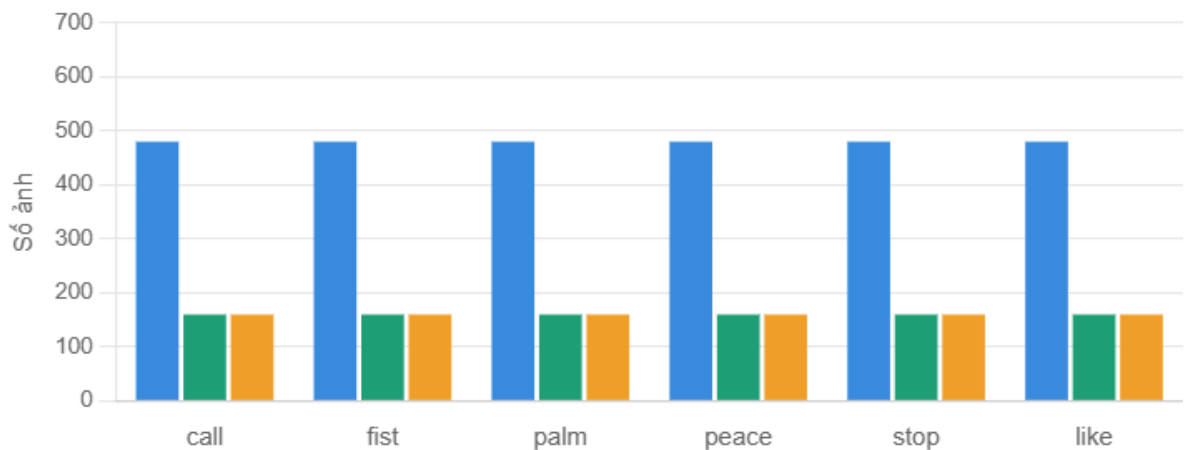
Tăng cường dữ liệu: Trong quá trình huấn luyện, Ultralytics tự động áp dụng tăng cường xử lý mặc định, tăng tính đa dạng và khả năng tổng quát hóa của mô hình.

- Lật ngang ảnh với xác suất 50%.
- Thay đổi độ sáng và độ tương phản.
- Ghép 4 ảnh thành một để mô hình học đồng thời nhiều bối cảnh.
- Cắt ngẫu nhiên với thay đổi tỷ lệ.

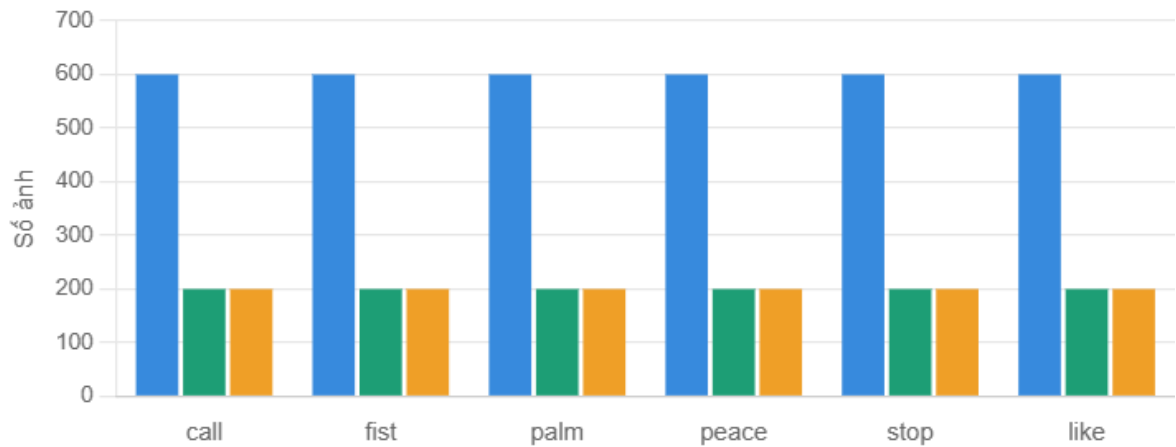
Định nghĩa các tập Train, Validation và Test:

- Tập Train: Là tập dữ liệu dùng để huấn luyện trực tiếp mô hình trong quá trình máy học.
- Tập Validation: Là tập các giá trị được dùng để giám sát và kiểm thử độ chính xác của mô hình máy học trong quá trình huấn luyện.
- Tập Test: Là tập dữ liệu dùng để kiểm tra sau khi mô hình đã học xong.

Cả hai lần huấn luyện đều chia dữ liệu theo tỷ lệ 60% - 20% - 20%. Phân chia dữ liệu được thực hiện ngẫu nhiên với seed cố định. Mô hình áp dụng kỹ thuật xáo trộn phân tầng đảm bảo không có ảnh nào xuất hiện đồng thời trong nhiều tập.



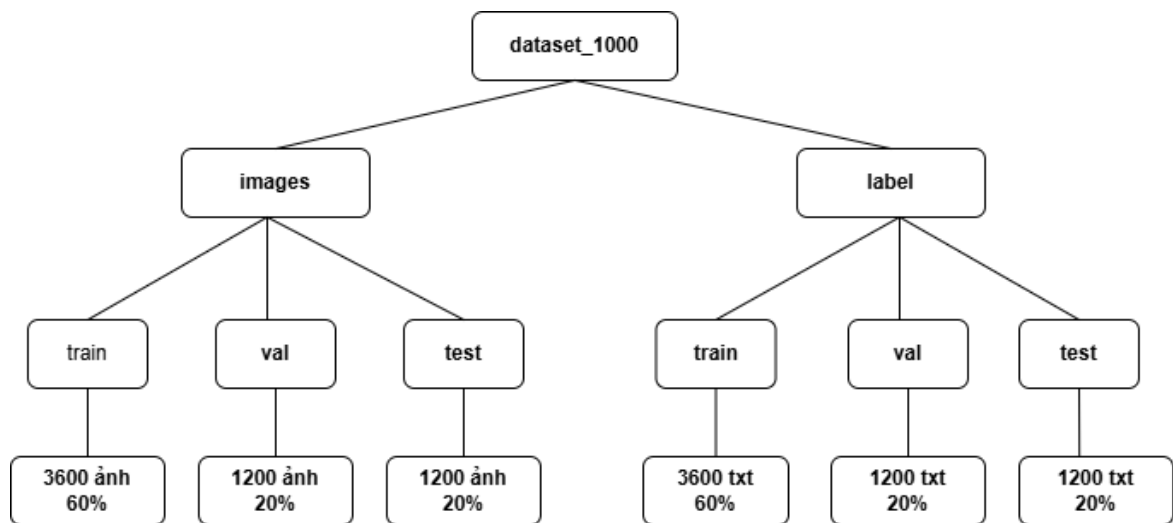
Hình 2.1 Tỷ lệ tập Train/Val/Test lần huấn luyện 1 với 4.800 ảnh (800 ảnh/lớp)



Hình 2.2 Tỷ lệ tập Train/Val/Test lần huấn luyện 2 với 6.000 ảnh (1.000 ảnh/lớp)

Mặc dù số lượng ảnh được chia đều nhưng trong một số ảnh của HaGRID có thể xuất hiện nhiều hơn một bàn tay hoặc một số ảnh có nhãn không đạt đã bị loại bỏ ở bước tiền xử lý. Do đó, biểu đồ phân bố dữ liệu thực tế sẽ đếm theo số lượng hộp giới hạn, dẫn đến sự chênh lệch nhỏ về mặt con số so với số lượng ảnh gốc.

Cấu trúc thư mục tuân theo chuẩn YOLO:



Hình 2.3 Cấu trúc thư mục YOLO

Nội dung file gesture.yaml:

```
path: /content/drive/MyDrive/KLTN_hand_gesture/dataset_1000

train: images/train
val: images/val
test: images/test

names:
  0: call
  1: fist
  2: palm
  3: peace
  4: stop
  5: like
```

Hình 2.4 Nội dung file gesture.yaml

2.3 Thiết kế hệ thống

2.3.1 Kiến trúc tổng thể của hệ thống

Hệ thống nhận diện được thiết kế theo kiến trúc mô-đun, tách biệt rõ giai đoạn huấn luyện và giai đoạn nhận diện thời gian thực. Hai giai đoạn kết nối với nhau duy nhất qua file best.pt.

Hệ thống gồm ba module chính:

Module xử lý dữ liệu: Các script chuẩn bị và chia tập dữ liệu (split_dataset.py). Sau đó, chuyển đổi nhãn JSON sang định dạng YOLO. Thư mục được tổ chức một cách riêng biệt.

Module huấn luyện và đánh giá mô hình: Thực hiện huấn luyện mô hình trên Google Colab GPU T4: Sử dụng file cấu hình gesture.yaml và trọng số yolo12m.pt. Lưu kết quả huấn luyện qua file best.pt, last.pt và các metrics (train_results.zip).

Module ứng dụng thời gian thực: File gesture_app.py chịu trách nhiệm nạp mô hình (best.pt), đọc luồng webcam qua OpenCV, thực hiện nhận diện và hiển thị kết quả.

2.3.2 Luồng xử lý dữ liệu trong hệ thống

Luồng xử lý dữ liệu được thiết kế tối ưu cho cả hai giai đoạn:

Giai đoạn huấn luyện

Script `split_dataset.py` đọc dữ liệu từ HaGRID, chuyển đổi nhãn và chia thành các thư mục `train/images`, `train/labels`, `val/images`, `val/labels`, `test/images`, `test/labels`.

File `gesture.yaml` định nghĩa đường dẫn và số lớp (`nc=6`).

Chạy lệnh huấn luyện Ultralytics để tạo ra `best.pt` và các file kết quả huấn luyện.

Giai đoạn nhận diện thời gian thực

Thu thập dữ liệu đầu vào: Webcam được khởi tạo bằng `cv2.VideoCapture(0)` trong file `gesture_app.py`. Mỗi frame thu được dưới dạng mảng NumPy.

Tiền xử lý: Ultralytics tự động chuẩn hóa kích thước về 640×640 pixel bằng kỹ thuật đệm viền và chuẩn hóa giá trị pixel về khoảng $[0, 1]$.

Suy luận mô hình: Khung hình được đưa vào mô hình đã load từ `best.pt`. YOLOv12m sẽ trả về các kết quả phát hiện bao gồm hộp giới hạn, `class_id` và độ tin cậy.

Hậu xử lý: Ultralytics tự động áp dụng thuật toán triệt tiêu phi cực đại với ngưỡng $\text{IoU} = 0.45$ loại bỏ các hộp giới hạn trùng lặp. Chỉ giữ lại các kết quả dự đoán có độ tin cậy ≥ 0.5 .

Hiển thị kết quả: Sử dụng hàm `results[0].plot()` để vẽ hộp giới hạn màu xanh lá, nhãn lớp kèm độ tin cậy và chỉ số FPS trực tiếp lên khung hình.

Vòng lặp và thoát: Vòng lặp `while True` tiếp tục xử lý khung tiếp theo. Nhấn phím “Q” để thoát chương trình, giải phóng camera và đóng cửa sổ hiển thị.

2.4 Xây dựng và huấn luyện mô hình YOLO

2.4.1 Cấu hình của mô hình

Đề tài sử dụng YOLOv12m (medium), được công bố đầu năm 2025 bởi Tian, Ye và Doermann [1]. Có ba lý do lựa chọn YOLOv12m: Bài toán 6 lớp cử chỉ với 6.000 ảnh có độ phức tạp trung bình, medium đạt độ chính xác cao hơn nano và small. Khi huấn luyện mô hình, GPU Tesla T4 thích hợp kích thước lô 16, không gây lỗi hết bộ nhớ. Đồng thời, trọng số `yolo12m.pt` trên COCO đã sẵn có.

Bảng 2.2 Cấu hình mô hình

Thông số	Giá trị
Backbone	R-ELAN kết hợp cơ chế Area Attention (A^2)
Neck	FPN-PAN
Head	Decoupled anchor-free
Đầu vào	Ảnh 640×640 pixel
Đầu ra	6 cử chỉ tay

2.4.2 Thiết lập tham số huấn luyện

Quá trình huấn luyện được thực hiện qua hai giai đoạn trên nền tảng Google Colab sử dụng GPU Tesla T4. Chiến lược fine-tuning được áp dụng nhằm cải thiện dần hiệu năng mô hình.

Lần huấn luyện 1: Mô hình được khởi tạo từ trọng số đã được huấn luyện trước trên COCO nhằm học các đặc trưng cơ bản của 6 lớp cử chỉ tay. Hệ thống sử dụng tập dữ liệu quy mô 4.800 ảnh (800 ảnh/lớp).

Bảng 2.3 Tham số lần huấn luyện 1

Tham số	Giá trị
Mô hình nền	yolov12m.pt (pretrained COCO)
Số epochs	50
imgsz	640
Batch size	16
Workers	2
Cache	False
Optimizer	Auto
Run name	train
Thư mục lưu	/content/runs/detect/train
Nền tảng	Google Colab (GPU Tesla T4)

Lần huấn luyện 2: Mô hình được tinh chỉnh từ trọng số tốt nhất (best.pt) của lần huấn luyện 1 nhằm cải thiện hiệu năng và giảm nhòe lẫn giữa các lớp. Lúc này, tập dữ liệu được mở rộng lên 6.000 ảnh (1.000 ảnh/lớp) để tăng cường tính đa dạng.

Bảng 2.4 Tham số lần huấn luyện 2

Tham số	Giá trị
Mô hình nền	best.pt (từ lần huấn luyện 1)
Số epochs	40
imgsz	640
Batch size	16
Workers	2
Cache	False
Optimizer	Auto
Run name	gesture_v2_L2
Thư mục lưu	/content/runs/detect/gesture_v2_L2
Nền tảng	Google Colab (GPU Tesla T4)

Các siêu tham số chính của thuật toán tối ưu hóa:

Tốc độ học ban đầu - 0.01: Quyết định mức độ cập nhật trọng số ở mỗi bước. Giá trị 0.01 giúp mô hình hội tụ nhanh, không bị trượt khỏi điểm tối ưu.

Động lượng - 0.937: Tận dụng quán tính từ các bước học trước để tăng tốc độ hội tụ và giảm dao động trong quá trình huấn luyện.

Suy giảm trọng số - 0.0005: Kỹ thuật điều chuẩn ngăn chặn hiện tượng quá khớp bằng cách phạt các trọng số có giá trị quá lớn.

Khởi động - 3 chu kỳ: Giai đoạn tăng dần tốc độ học từ mức rất nhỏ lên 0.01 trong 3 chu kỳ đầu, mô hình sẽ tránh các cú sốc gradient và ổn định từ sớm.

2.4.3 Quá trình huấn luyện và đánh giá mô hình

Quy trình huấn luyện được triển khai trên Google Colab, dữ liệu lưu trữ trên Google Drive và được đồng bộ thông qua `drive.mount()`.

Lần huấn luyện 1

```
from ultralytics import YOLO

model = YOLO("/content/drive/MyDrive/KLTN_hand_gesture/yolo12m.pt")

model.train(
    data="/content/drive/MyDrive/KLTN_hand_gesture/gesture.yaml",
    epochs=50,
    imgsz=640,
    batch=16,
    workers=2
)
```

Hình 2.5 Tham số cấu hình lần huấn luyện 1

Lần huấn luyện 2

```
from ultralytics import YOLO

model = YOLO("/content/drive/MyDrive/KLTN_hand_gesture L2/yolo12m.pt")

model.train(
    data="/content/drive/MyDrive/KLTN_hand_gesture L2/gesture.yaml",
    epochs=80,
    imgsz=640,
    batch=16,
    workers=2,
    project="/content/drive/MyDrive/KLTN_hand_gesture L2/runs",
    name="gesture_v2_1000img"
)
```

Hình 2.6 Tham số cấu hình lần huấn luyện 2

Trong quá trình huấn luyện, hệ thống theo dõi ba hàm loss:

- box_loss: Sai số vị trí hộp giới hạn.
- cls_loss: Sai số phân loại.
- dfl_loss: Distribution Focal Loss.

Các chỉ số này được ghi nhận trên cả tập train và validation, cho phép phát hiện hiện tượng quá khớp khi validation loss tăng trong khi train loss giảm.

Sau mỗi chu kỳ, mô hình được đánh giá trên tập validation. Trọng số có giá trị mAP@50 cao nhất được lưu vào file: weights/best.pt. Sau khi hoàn tất cả hai giai đoạn huấn luyện, mô hình tốt nhất từ lần huấn luyện 2 được sử dụng để đánh giá chính thức: model.val(plots=True). Hệ thống tự động sinh ra:

- Confusion Matrix.
- Precision - Recall Curve.
- F1-score Curve.

2.5 Triển khai hệ thống nhận diện thời gian thực

2.5.1 Tích hợp mô hình vào hệ thống

File best.pt sau khi huấn luyện được nạp vào Python qua thư viện Ultralytics. Mô hình được khởi tạo một lần duy nhất khi khởi động ứng dụng và tái sử dụng toàn bộ vòng lặp nhận diện:

```
from ultralytics import YOLO
import torch

MODEL_PATH = r"D:\KLTN\hand-gesture-yolo12\runs\detect\gesture_v2_L2_all\weights\best.pt"
CLASS_NAMES = ["call", "fist", "palm", "peace", "stop", "like"]
```

Hình 2.7 Tích hợp best.pt vào hệ thống nhận diện

FPS sẽ thấp hơn đáng kể khi sử dụng trên máy không có GPU.

2.5.2 Xử lý video/camera và hiển thị kết quả

Video từ webcam được thu nhận và xử lý thời gian thực qua thư viện OpenCV trong vòng lặp chính của ứng dụng. Camera được khởi tạo bằng:

```
self.cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, INFER_W)
self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, INFER_H)

self.timer = QTimer()
self.timer.timeout.connect(self.update_frame)
```

Hình 2.8 Lệnh khởi tạo webcam

Tham số CAP_DSHOW tối ưu tốc độ truy cập camera. Kích thước khung hình đầu vào được thiết lập 320×180 giảm độ trễ và tăng tốc độ xử lý.

Cơ chế xử lý khung: Sử dụng QTimer định kỳ đọc khung với chu kỳ 30ms (tương đương ~33 FPS). Khung sau khi đọc được chuyển sang một luồng riêng để xử lý suy luận, tránh xung đột với luồng giao diện người dùng, đảm bảo giao diện không bị giật.

Tối ưu suy luận mô hình: Ảnh được chuẩn hóa về kích thước nhỏ hơn trước khi đưa vào mô hình, giảm chi phí tính toán. Quá trình dự đoán sử dụng ngưỡng tin cậy $\text{conf} = 0.45$. Tận dụng GPU để tăng tốc suy luận nếu phần cứng hỗ trợ.

Làm mượt kết quả: Kết quả suy luận (hộp giới hạn, nhãn) được quy đổi kích thước ngược về kích thước ảnh gốc đảm bảo độ chính xác khi hiển thị. Áp dụng cơ chế làm mượt bằng cách lấy nhãn xuất hiện nhiều nhất trong một số khung gần nhất, kết quả nhận diện ổn định hơn, giảm nhiễu.

Hiển thị kết quả: Hộp giới hạn vẽ trực tiếp lên khung và hiển thị trên giao diện:

- Hộp giới hạn màu xanh
- Nhãn lớp và độ tin cậy.

FPS được tính theo công thức và hiển thị liên tục theo thời gian thực bên góc phải màn hình:

$$FPS = \frac{1}{\text{Thời gian kết thúc frame} - \text{Thời gian bắt đầu frame}}$$

CHƯƠNG 3: THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương 3 trình bày chi tiết về môi trường, các kịch bản thực nghiệm và phân tích kết quả đạt được. Hiệu năng của mô hình được đánh giá toàn diện thông qua các chỉ số định lượng (mAP, Precision, Recall) và ma trận nhầm lẫn nhằm đối chiếu sự cải thiện giữa hai lần huấn luyện. Bên cạnh đó, chương cũng đánh giá tính ổn định, tốc độ xử lý (FPS) của hệ thống trong điều kiện thời gian thực và minh họa trực quan khả năng ứng dụng mô hình để điều khiển thiết bị trong môi trường mô phỏng 3D.

3.1 Môi trường thực nghiệm

3.1.1 Phần cứng và phần mềm sử dụng

Quá trình thực nghiệm được thực hiện trên hai môi trường tách biệt tương ứng với hai giai đoạn của hệ thống.

Môi trường huấn luyện - Google Colab

Bảng 3.1 Môi trường huấn luyện

Thành phần	Thông số
Nền tảng	Google Colaboratory
GPU	NVIDIA Tesla T4 - 16 GB VRAM
RAM hệ thống	12,7 GB
Hệ điều hành	Ubuntu 22.04 (môi trường Colab)
Python	3.12.13
Ultralytics	8.4.33
PyTorch	2.10.0+cu128
Lưu trữ	Google Drive

Môi trường kiểm thử và demo thực tế

Bảng 3.2 Môi trường kiểm thử thực tế

Thành phần	Thông số
CPU	Intel Core i5-12500H (thế hệ 12)
GPU	Không sử dụng (CPU-only)

RAM	8 GB
Hệ điều hành	Windows 11
Python	3.10.12
Webcam	Độ phân giải tối thiểu 640×480 pixel
Thư viện chính	Ultralytics, OpenCV-Python, PyQt6, PyTorch

Các công cụ sử dụng trong toàn bộ quá trình thực nghiệm:

Google Colab: Môi trường huấn luyện và đánh giá chính, cung cấp GPU Tesla T4 miễn phí [9].

Ultralytics YOLO Python API: Thư viện chính cho huấn luyện, đánh giá và suy luận [10].

OpenCV (opencv-python): Xử lý luồng video từ webcam và biến đổi ảnh [11].

PyQt6: Framework xây dựng giao diện đồ họa đa luồng cho ứng dụng demo [12].

LabelImg: Công cụ gán nhãn bounding box định dạng YOLO, dùng trong giai đoạn chuẩn bị dữ liệu.

3.1.2 Thiết lập thực nghiệm

Quá trình thực nghiệm được chia thành hai lần huấn luyện độc lập trên nền tảng Google Colab, với quy mô dữ liệu được mở rộng qua từng giai đoạn với mục đích tối ưu hóa hiệu năng của mô hình:

- Lần huấn luyện 1: Sử dụng tập dữ liệu gồm 4.800 ảnh (800 ảnh cho mỗi lớp). Ở giai đoạn này, mô hình được huấn luyện để học các đặc trưng nhận diện cơ bản.
- Lần huấn luyện 2: Tập dữ liệu được bổ sung và nâng quy mô lên 6.000 ảnh (1.000 ảnh cho mỗi lớp). Hệ thống sử dụng trọng số tốt nhất đạt được từ phiên huấn luyện một để tiếp tục quá trình tinh chỉnh, giúp mô hình khắc phục các nhầm lẫn và tăng độ chính xác.

Trước mỗi phiên, tập dữ liệu được sao chép từ Google Drive vào bộ nhớ cục bộ của Colab để tăng tốc độ nạp dữ liệu trong quá trình huấn luyện. Trọng số mô hình

best.pt và toàn bộ nhật ký kết quả được tự động lưu ngược về Google Drive đảm bảo an toàn dữ liệu khi phiên Colab hết hạn. Tập dữ liệu kiểm thử (chiếm 20% tổng số lượng ảnh, tương ứng 960 ảnh ở lần huấn luyện 1 và 1.200 ảnh ở lần huấn luyện 2) được tách biệt hoàn toàn trước khi bắt đầu mỗi phiên. Tập dữ liệu này được cô lập hoàn toàn và không tham gia vào bất kỳ bước huấn luyện hay điều chỉnh tham số nào. Hệ thống ánh xạ trực tiếp các cử chỉ tay thành các lệnh điều khiển thiết bị trong môi trường mô phỏng 3D:

Bảng 3.3 Thao tác điều khiển ứng cử cử chỉ tay

STT	Cử chỉ tay	Thao tác điều khiển
1	Call	CALL_NURSE
2	Fist	CLOSE_DOOR
3	Palm	OPEN_DOOR
4	Peace	TOGGLE_BED
5	Stop	TOGGLE_LIGHT
6	Like	MOVE_FORWARD / RIGHT / LEFT

Riêng với cử chỉ Like, hệ thống phân biệt ba hành động điều khiển dựa trên vị trí ngang của hộp giới hạn so với khung hình camera. Cụ thể, khung hình camera được chia làm ba dải dọc bằng nhau. Khi tâm hộp giới hạn rơi vào 1/3 khung hình bên trái, lệnh MOVE_LEFT được thực thi. Khi tâm hộp giới hạn rơi vào 1/3 khung hình bên phải, lệnh MOVE_RIGHT được thực thi. Vùng trung tâm còn lại sẽ kích hoạt lệnh MOVE_FORWARD.

3.2 Kịch bản thử nghiệm

3.2.1 Mục tiêu thử nghiệm và tiêu chí đánh giá hệ thống

Mục tiêu: Đánh giá toàn diện hệ thống trên ba khía cạnh:

Đánh giá định lượng trên tập dữ liệu kiểm thử: Đo Precision, Recall, F1-score, mAP@50 và mAP@50-95 trên toàn bộ tập ảnh test tương ứng của mỗi lần huấn luyện (960 ảnh đối với mô hình huấn luyện lần 1 và 1.200 ảnh đối với mô hình huấn luyện lần 2). Phân tích từng lớp cử chỉ và xác định lớp nào mô hình nhận diện tốt và lớp nào còn hạn chế.

Phân tích confusion matrix: Xác định cụ thể các cặp lớp hay nhầm lẫn với nhau. Từ đó, suy ra nguyên nhân sai sót của mô hình.

Đánh giá hiệu suất thời gian thực: Đo FPS thực tế trên máy tính cá nhân CPU khi chạy ứng dụng PyQt6 và đánh giá độ ổn định nhận diện và phản hồi liên tục trong điều kiện thực tế.

Bộ dữ liệu kiểm thử (test set) chiếm 20% tổng dữ liệu được trích xuất từ nguồn dữ liệu gốc HaGRID [7]. Số lượng ảnh kiểm thử được phân bố đều trên 6 lớp cử chỉ (call, fist, palm, peace, stop và like) cụ thể như sau: 160 ảnh/lớp (20% của 800 ảnh, áp dụng cho lần huấn luyện 1) và 200 ảnh/lớp (20% của 1.000 ảnh, áp dụng cho lần huấn luyện 2). Mô hình được đảm bảo chưa từng tiếp xúc với bất kỳ hình ảnh nào trong tập kiểm thử này ở giai đoạn huấn luyện. Ảnh giữ nguyên độ phân giải gốc 1920×1080 pixel từ HaGRID và được tự động chuẩn hóa kích thước về 640×640 khi đưa vào model.val().

Hệ thống được đánh giá theo các tiêu chí:

Precision và Recall: Đánh giá riêng từng lớp cử chỉ, xác định lớp nào mô hình nhận tốt nhất và lớp nào còn nhầm nhiều nhất.

mAP@50: Chỉ số đánh giá tổng hợp chính với ngưỡng IoU = 0,5. Theo kỳ vọng, mô hình phải đạt tối thiểu 80% để được xem là đáp ứng bài toán.

mAP@50-95: Đánh giá chất lượng định vị hộp giới hạn tại 10 ngưỡng IoU từ 0,5 đến 0,95.

FPS thực tế: Yêu cầu tối thiểu là 10 FPS đảm bảo trải nghiệm thời gian thực chấp nhận được.

Confusion Matrix: Ma trận nhầm lẫn giữa các lớp, phân tích định tính nguyên nhân sai sót của mô hình.

3.2.2 Kịch bản và quy trình thực nghiệm

Các kịch bản thực nghiệm:

Kịch bản 1 - Đánh giá trên tập test: Tiến hành đánh giá độc lập hai trọng số best.pt thu được từ phiên huấn luyện 1 và phiên huấn luyện 2 bằng lệnh model.val() trên tập kiểm thử tương ứng của mỗi phiên (960 ảnh đối với lần huấn luyện 1 và 1.200 ảnh đối với lần huấn luyện 2). Mục tiêu trích xuất được các chỉ số mAP, Precision, Recall và ma trận nhầm lẫn cho từng lớp.

Kịch bản 2 - Nhận diện thời gian thực qua webcam: Nạp trọng số mô hình tối ưu nhất từ lần huấn luyện 2 vào `gesture_app.py` và chạy thử nghiệm trên máy tính cá nhân Windows 11 CPU i5, kết nối webcam, thực hiện nhận diện liên tục trong điều kiện ánh sáng phòng bình thường. Lần lượt thực hiện từng cử chỉ trước camera, quan sát nhãn hiển thị, độ tin cậy và FPS trung bình ghi nhận trên giao diện.

Kịch bản 3 - Kiểm tra độ bền vững: Thử nghiệm nhận diện trong điều kiện thay đổi có chủ đích: góc nghiêng bàn tay (30° , 45° , 90°), thay đổi khoảng cách (gần $\sim 30\text{cm}$, xa $\sim 150\text{cm}$), nền phức tạp (nhiều vật thể phía sau), thiếu sáng nhẹ và sử dụng tay trái thay vì tay phải. Mục tiêu xác định ngưỡng điều kiện mà hệ thống còn hoạt động ổn định.

Thực nghiệm được tiến hành theo năm bước:

Bước 1: Hoàn tất huấn luyện 1 (50 epochs) và 2 (40 epochs) trên Google Colab GPU T4. Lưu `best.pt` của L2 về Google Drive.

Bước 2: Trên Colab, chạy `model.val()` lần lượt với `split='val'` và `split='test'` để thu thập đầy đủ chỉ số và xuất biểu đồ. Lưu toàn bộ kết quả vào thư mục `runs/` trên Drive.

Bước 3: Tải `best.pt` về máy tính cá nhân Windows 11. Cài đặt môi trường: `pip install ultralytics opencv-python pyqt6 torch`.

Bước 4: Chạy `gesture_app.py`, thực hiện kịch bản 2 và 3. Ghi nhận FPS trung bình hiển thị trên giao diện và quan sát chất lượng nhận diện của từng cử chỉ.

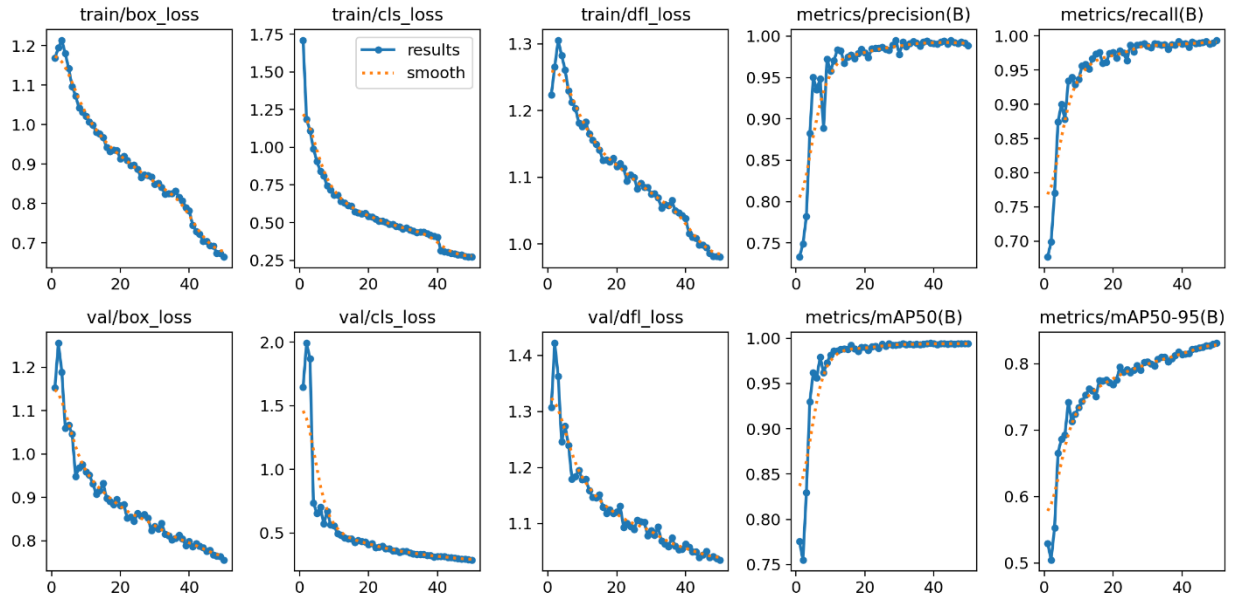
Bước 5: Tổng hợp toàn bộ số liệu, lập bảng so sánh, phân tích kết quả và đối chiếu với các kỳ vọng đặt ra ban đầu.

3.3 Kết quả huấn luyện

3.3.1 Kết quả quá trình huấn luyện lần 1

a. Phân tích biểu đồ loss, Precision, Recall và ma trận nhầm lẫn

Trong lần huấn luyện đầu tiên, mô hình YOLOv12m được khởi tạo từ trọng số pretrained trên tập dữ liệu COCO và huấn luyện trong 50 chu kỳ. Kết quả huấn luyện được thể hiện thông qua các biểu đồ loss, Precision, Recall và confusion matrix.

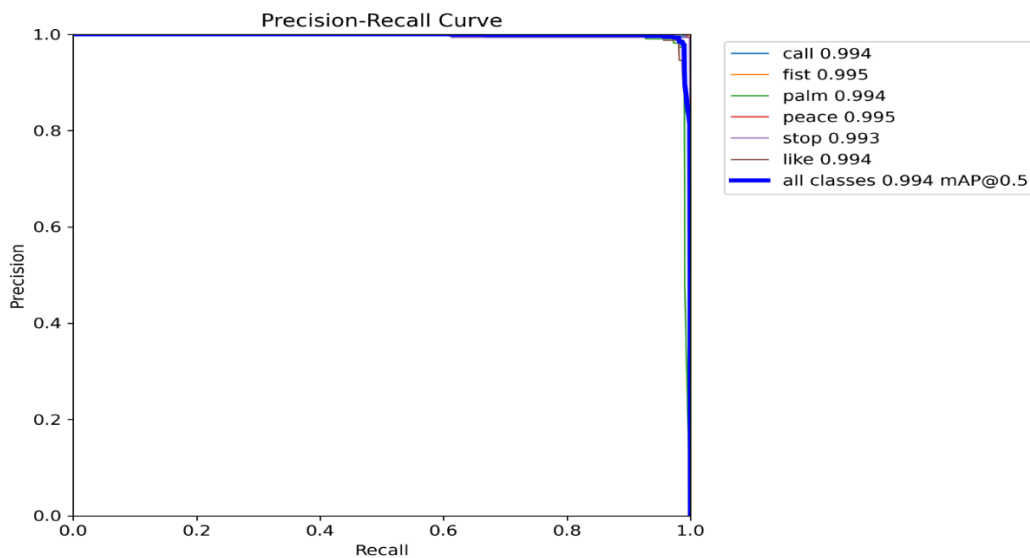


Hình 3.1 Biểu đồ quá trình huấn luyện (loss, precision, recall, mAP)

Các giá trị box_loss, cls_loss và df_l_loss đều giảm dần theo số chu kỳ. Chứng tỏ mô hình học được đặc trưng của dữ liệu và hội tụ tốt.

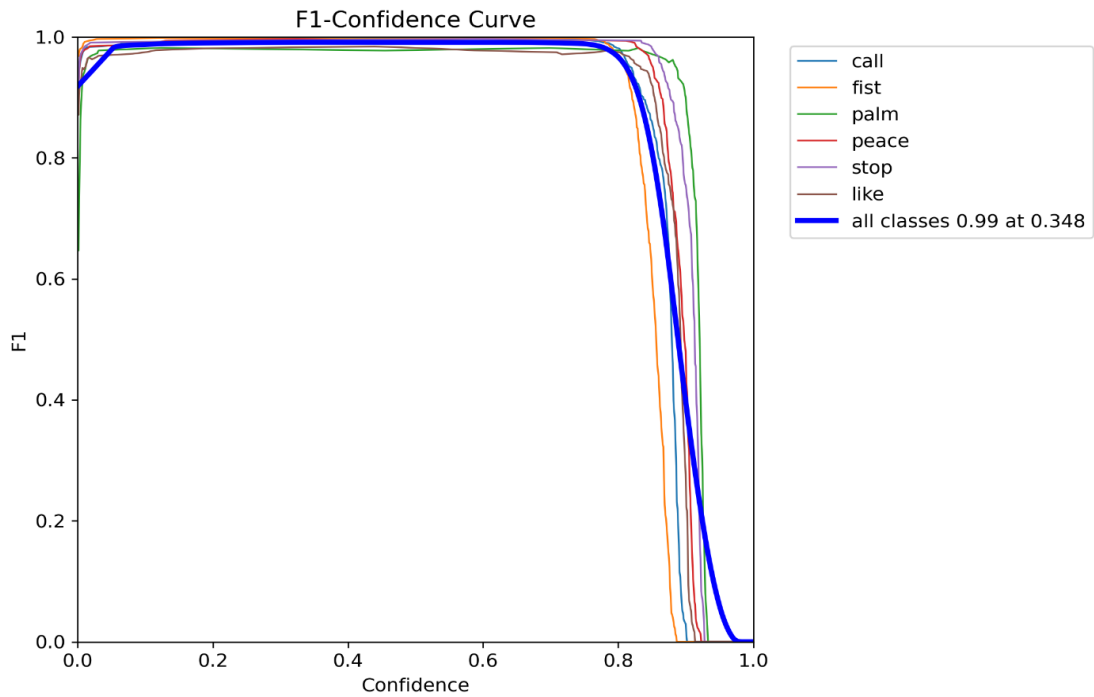
Precision và Recall tăng nhanh và ổn định ở mức cao ($\sim 0.98 - 1.0$). mAP@50 đạt xấp xỉ 0.99. mAP@50-95 đạt khoảng 0.83. Mô hình có khả năng phát hiện và định vị đối tượng chính xác.

b. Phân tích các đường cong đánh giá



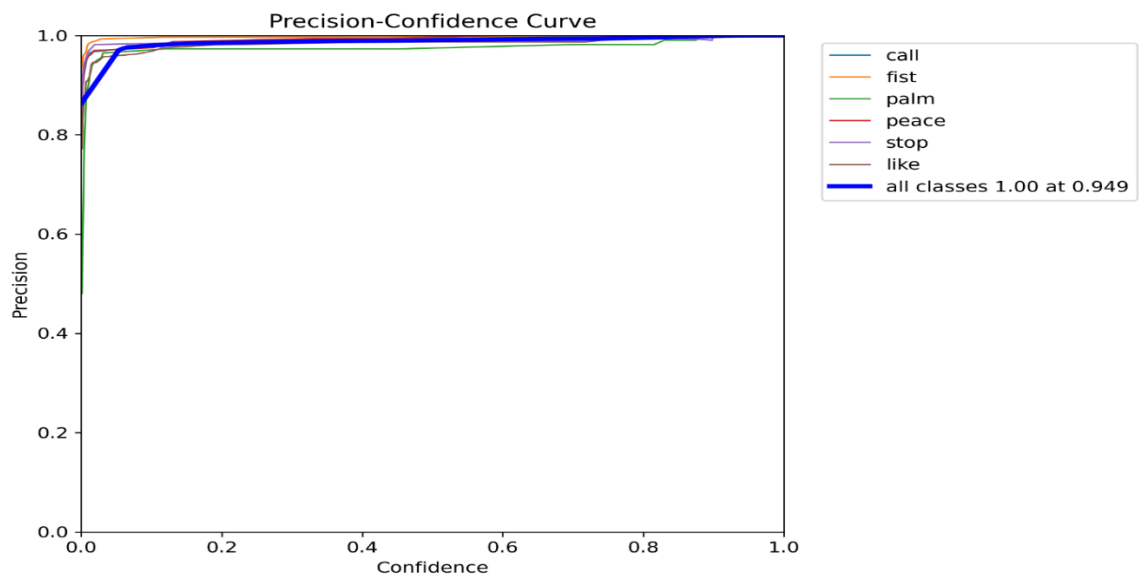
Hình 3.2 Đường cong Precision - Recall

Đường cong Precision - Recall cho thấy diện tích dưới đường cong rất lớn (gần 1). mAP@50 = 0.994. Mô hình đạt hiệu năng rất cao trên tập validation.



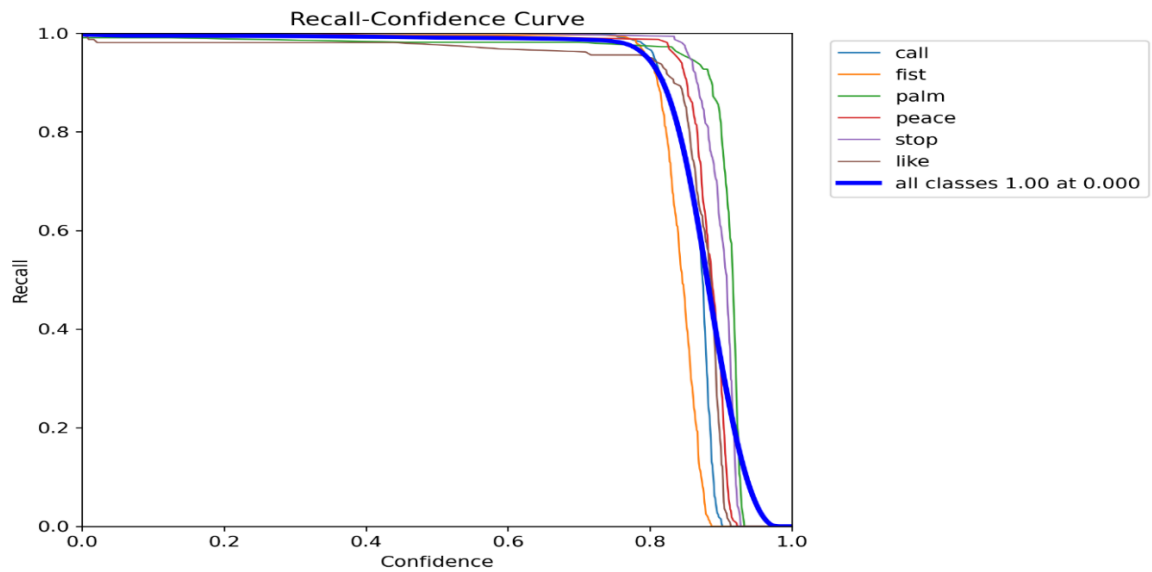
Hình 3.3 Đường cong F1 - Confidence

F1-score đạt giá trị cao nhất (~ 0.99) tại ngưỡng độ tin cậy khoảng 0.35. Ngưỡng độ tin cậy tối ưu để cân bằng giữa Precision và Recall là khoảng 0.3 - 0.4.



Hình 3.4 Đường cong Precision - Confidence

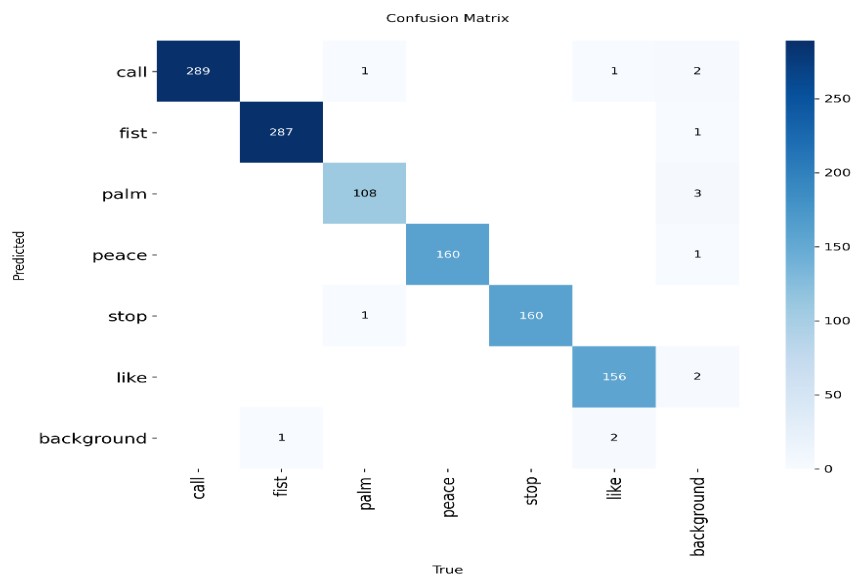
Precision tăng nhanh và đạt gần 1.0 ở độ tin cậy cao, cho thấy mô hình rất ít dự đoán sai khi ngưỡng độ tin cậy lớn.



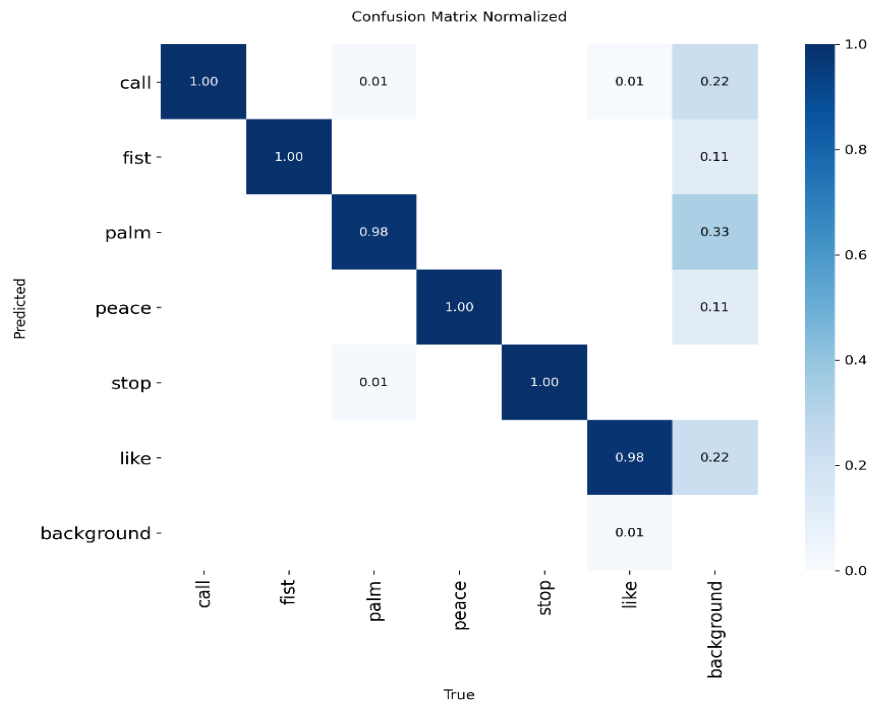
Hình 3.5 Đường cong Recall - Confidence

Recall giảm dần khi độ tin cậy tăng, phù hợp với lý thuyết: khi mô hình “khắt khe” hơn thì sẽ bỏ sót một số đối tượng.

c. Phân tích confusion matrix



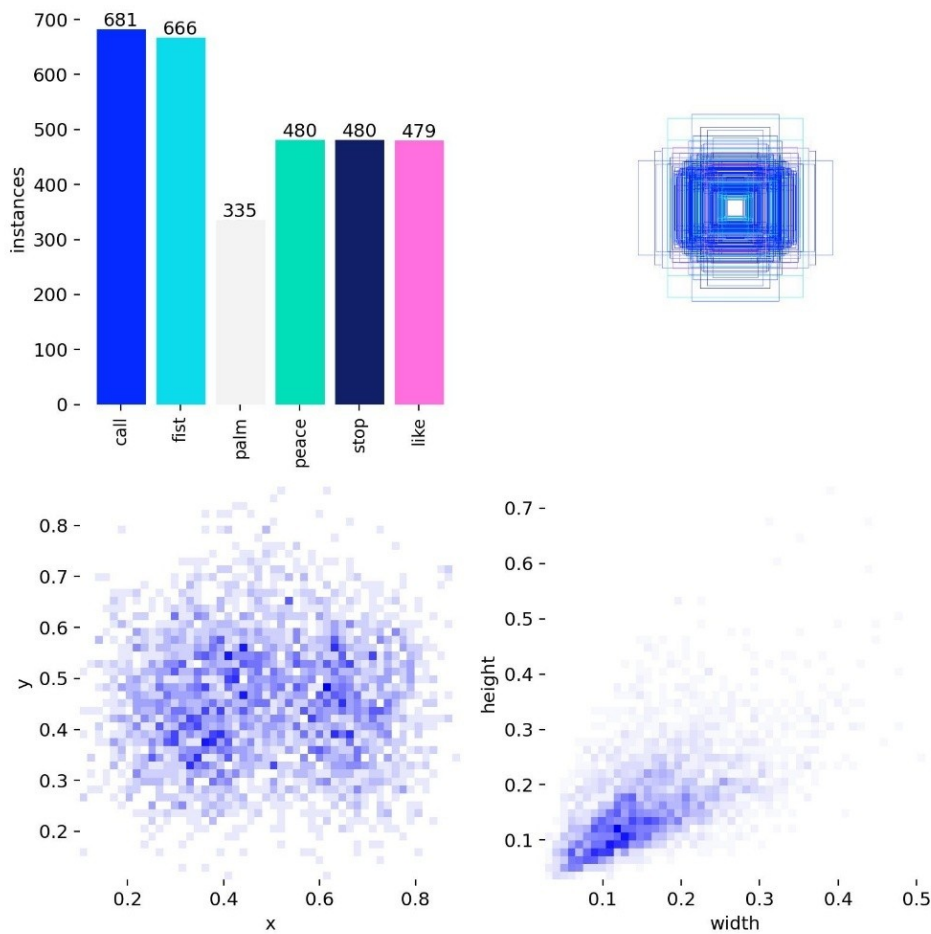
Hình 3.6 Confusion matrix của mô hình



Hình 3.7 Confusion matrix chuẩn hóa

Kết quả phản ánh hiệu năng dự đoán trên tập kiểm thử (Validation/Test). Con số trong ma trận đại diện cho số lượng bàn tay được phát hiện, không phải số lượng bức ảnh. Khi phân tích hơn 1100 mẫu hợp lệ, phần lớn đều hội tụ trên đường chéo chính của ma trận, cho thấy độ chính xác rất cao. Các lớp call, fist, peace và stop gần như không có bất kỳ sai sót nào. Dữ liệu thực tế có sự chênh lệch nhẹ (lớp palm có 108 cá thể, lớp call lên tới 289 cá thể), tỷ lệ nhận diện nhầm giữa các lớp chỉ ở mức vô cùng nhỏ (khoảng 1 - 2%). Mô hình có khả năng trích xuất đặc trưng mạnh mẽ, nhận diện tốt ngay cả các lớp thiểu số, không bị ảnh hưởng bởi sự chênh lệch số lượng mẫu đầu vào.

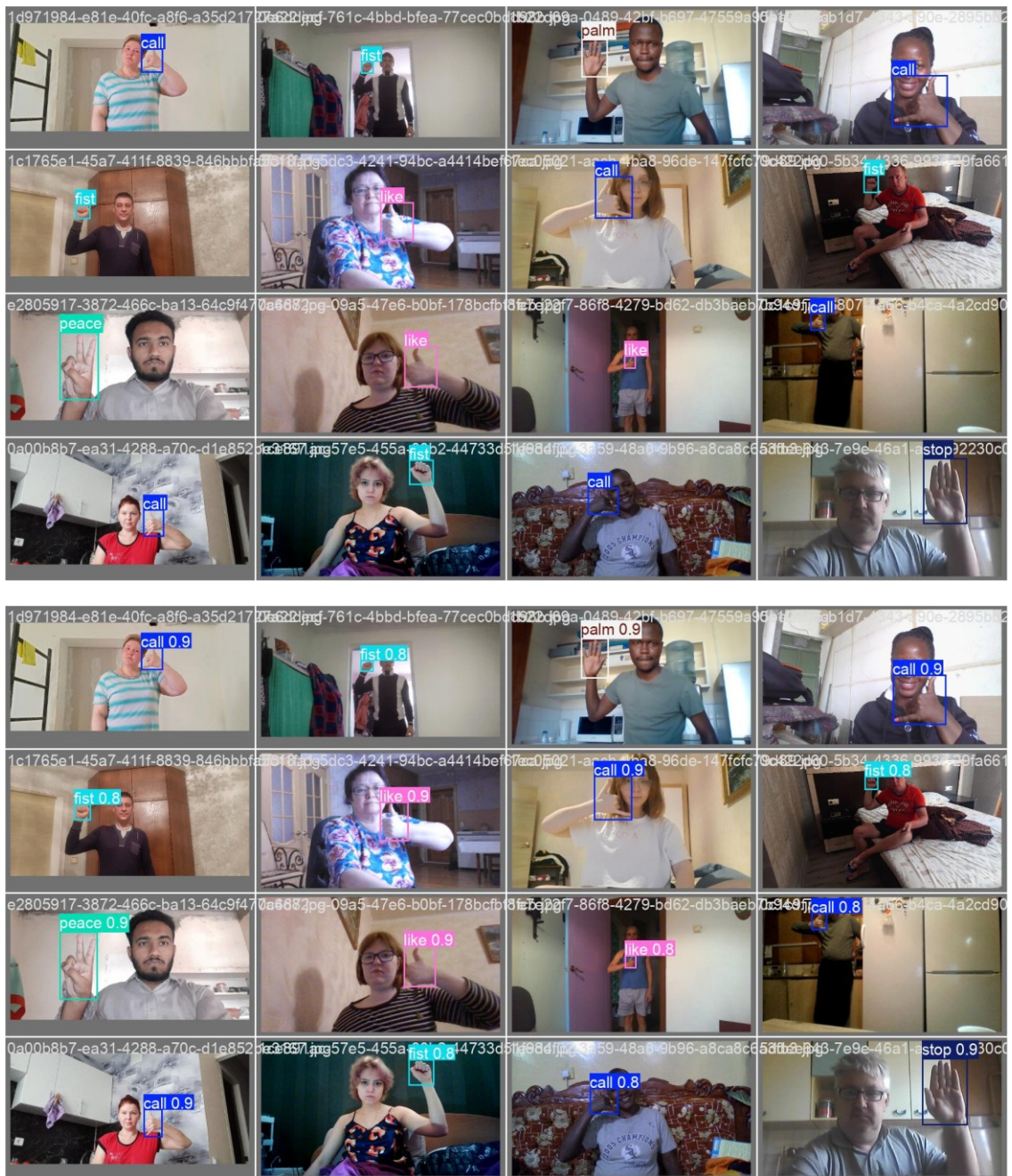
d. Phân tích dữ liệu và phân bố nhãn



Hình 3.8 Phân bố dữ liệu và bounding box trong tập huấn luyện

Biểu đồ phân bố thống kê dựa trên số lượng cá thể hộp giới hạn thực tế có trong tập huấn luyện. Cụ thể, call (681), fist (666), peace (480), stop (480), like (479) và palm (335). Sự chênh lệch này phản ánh đúng bản chất của dữ liệu thực tế khi số lượng bàn tay xuất hiện trong các khung hình không hoàn toàn đồng đều. Tuy nhiên, sự phân bố này vẫn nằm trong ngưỡng an toàn, độ lệch không quá lớn (ngoại trừ lớp palm ít hơn nhưng vẫn đủ mẫu đại diện), mô hình sẽ không bị lệch quá mức vào một lớp cụ thể. Biểu đồ tọa độ cho thấy các hộp giới hạn tập trung chủ yếu ở vùng trung tâm khung hình, phù hợp đặc tính người dùng giơ tay trước webcam trong bài toán nhận diện cử chỉ.

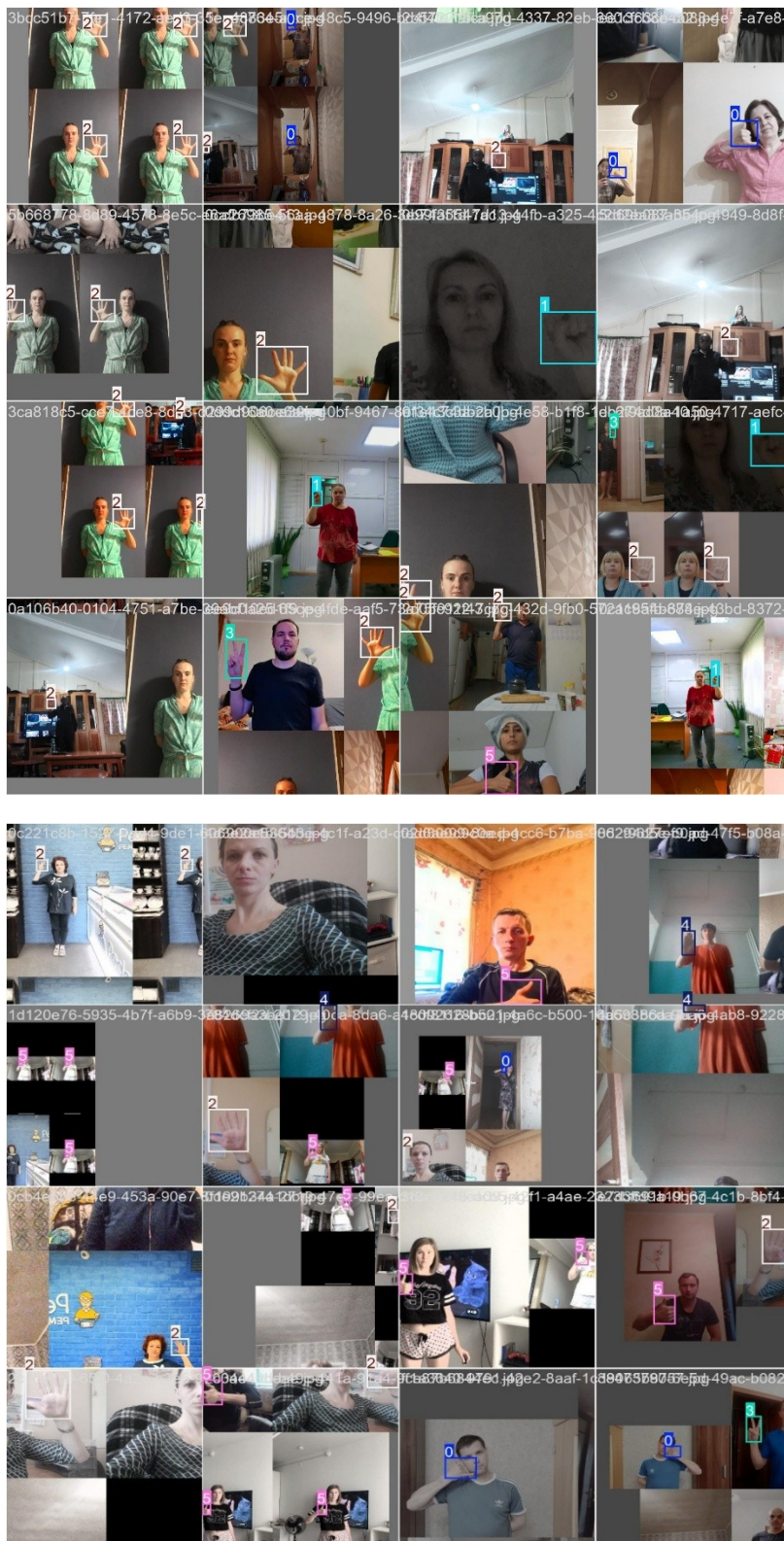
e. Kết quả dự đoán trên tập Validation

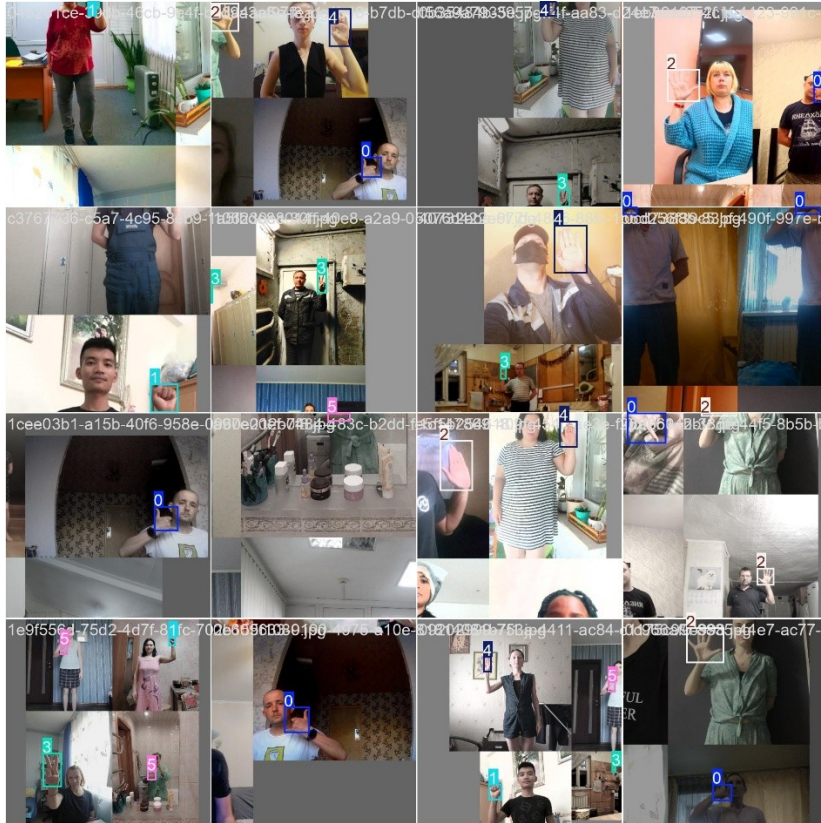


Hình 3.9 So sánh ground truth và kết quả dự đoán

Kết quả cho thấy mô hình có khả năng phát hiện chính xác vị trí bàn tay và phân loại đúng cử chỉ với độ tin cậy cao.

f. Minh họa quá trình huấn luyện





Hình 3.10 Một số batch dữ liệu trong quá trình huấn luyện

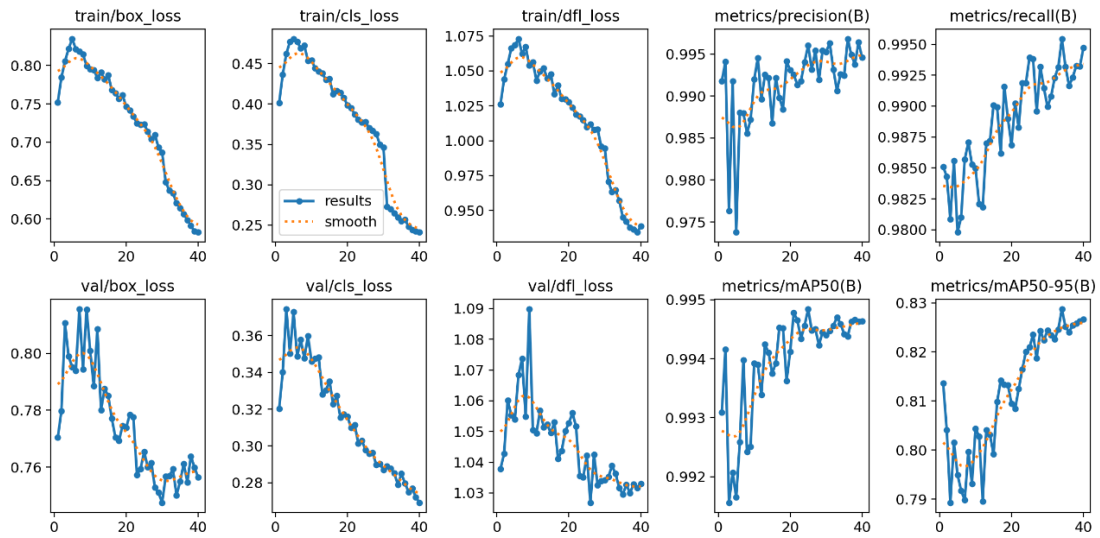
Các hình trên minh họa dữ liệu đầu vào và nhãn trong quá trình training, cho thấy dữ liệu đa dạng về môi trường, ánh sáng và góc nhìn.

Qua thực nghiệm ở lần huấn luyện 1, hệ thống dù đạt độ chính xác cao nhưng vẫn bộc lộ hạn chế về khả năng nhận diện các cử chỉ có góc nghiêng lớn và sự nhầm lẫn giữa các lớp cử chỉ tương đồng (palm và like). Đồng thời, tập dữ liệu 4.800 ảnh ban đầu chưa đủ đảm bảo tính ổn định trong các điều kiện môi trường khắc nghiệt.

3.3.2 Kết quả quá trình huấn luyện lần 2

Sau khi thực hiện huấn luyện lần 1 và thấy còn tồn tại hạn chế, mô hình được tiếp tục huấn luyện lần 2: Nâng quy mô lên 6.000 ảnh để tăng tính đa dạng và khả năng học đặc trưng của mô hình. Tối ưu hóa trọng số: Sử dụng kỹ thuật tinh chỉnh từ kết quả lần 1 để mô hình hội tụ sâu hơn vào các lớp dữ liệu khó, từ đó nâng mAP@50 lên mức tối ưu (~0.994) và đảm bảo tính ổn định khi triển khai thực tế.

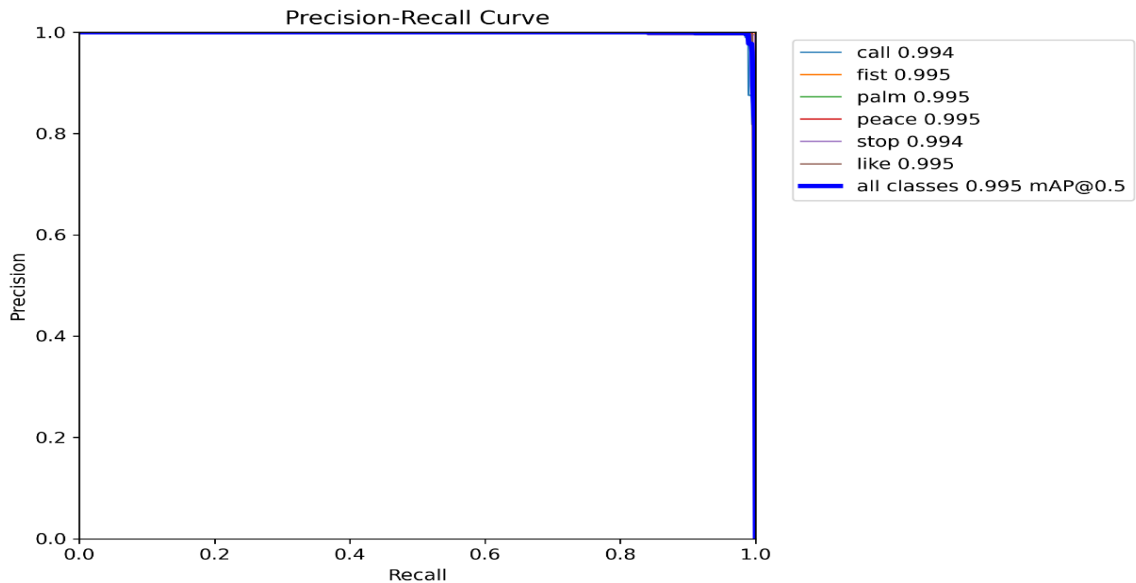
a. Phân tích biểu đồ *loss*, *Precision*, *Recall* và ma trận nhầm lẫn



Hình 3.11 Biểu đồ quá trình huấn luyện (*loss*, *precision*, *recall*, *mAP*)

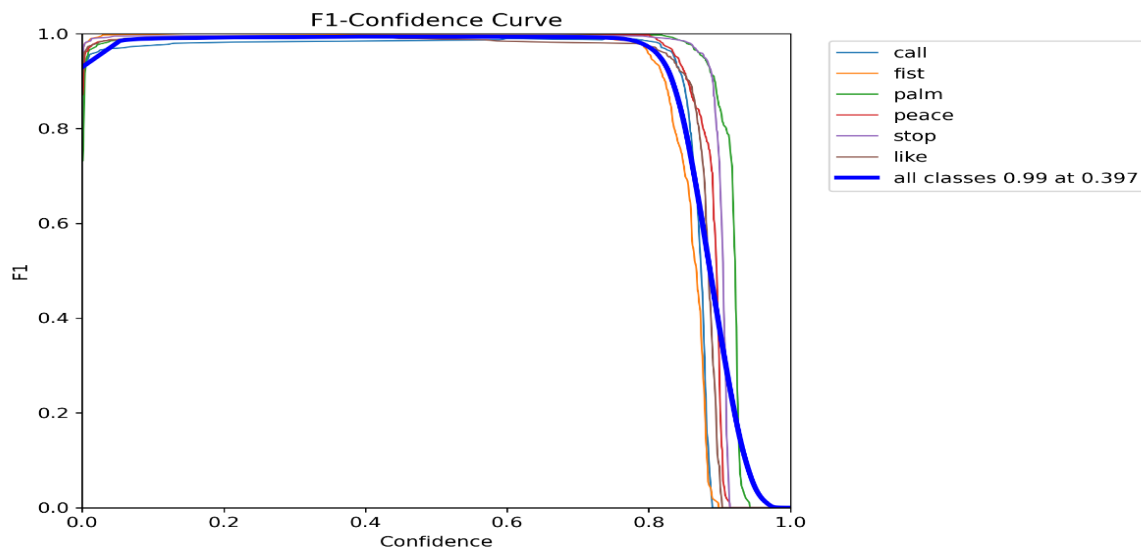
Các giá trị *box_loss*, *cls_loss* và *df_l_loss* tiếp tục giảm đều theo số chu kỳ và ổn định hơn so với lần huấn luyện đầu tiên. Mô hình hội tụ tốt và học được đặc trưng dữ liệu hiệu quả hơn. *Precision* và *Recall* đạt gần 1.0 và ổn định từ sớm. $mAP@50 \approx 0.995$. $mAP@50-95 \approx 0.83 - 0.84$. Mô hình đạt hiệu năng rất cao và cải thiện nhẹ so với lần huấn luyện trước.

b. Phân tích các đường cong đánh giá

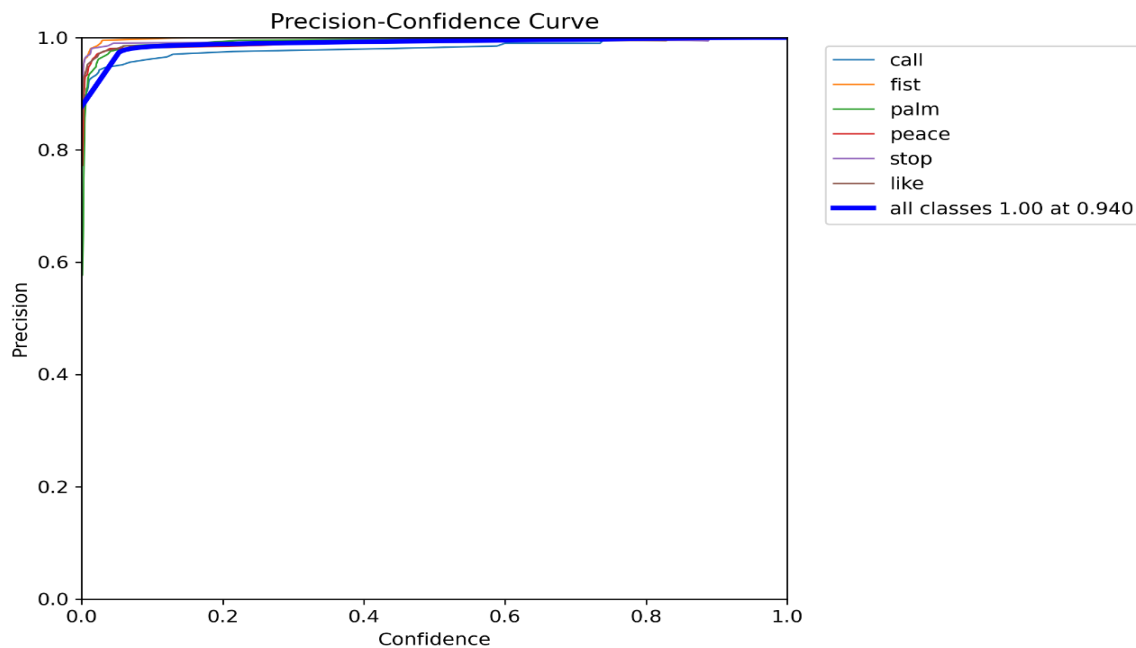


Hình 3.12 Đường cong *Precision* - *Recall*

Đường cong Precision - Recall có diện tích rất lớn, gần như bao phủ toàn bộ vùng phía trên. $mAP@50 = 0.995$. Mô hình đạt hiệu năng cao trên tập Validation.

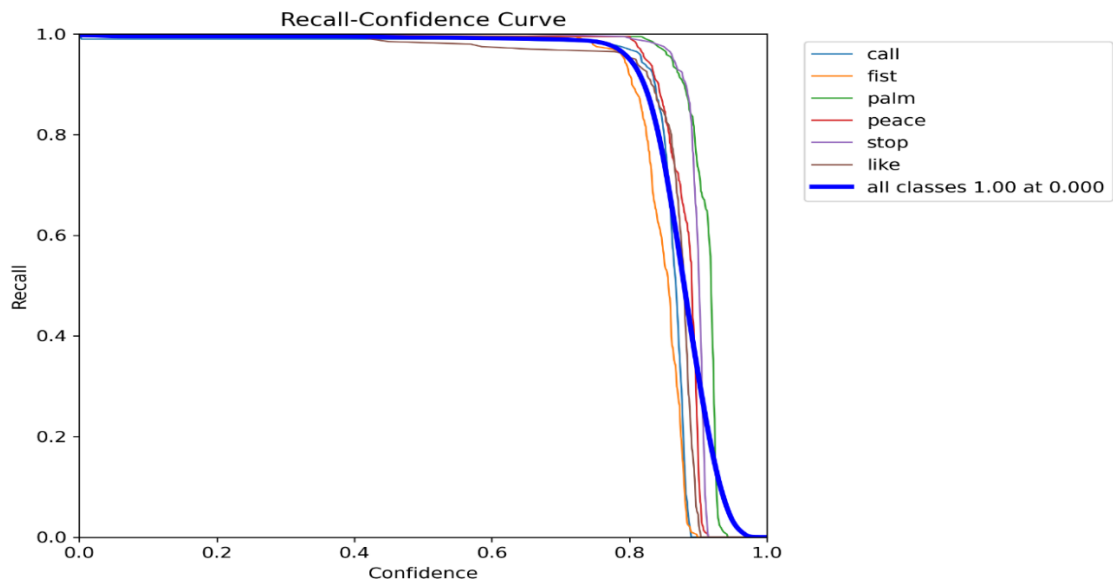


Hình 3.13 Đường cong F1 - Confidence



Hình 3.14 Đường cong Precision - Confidence

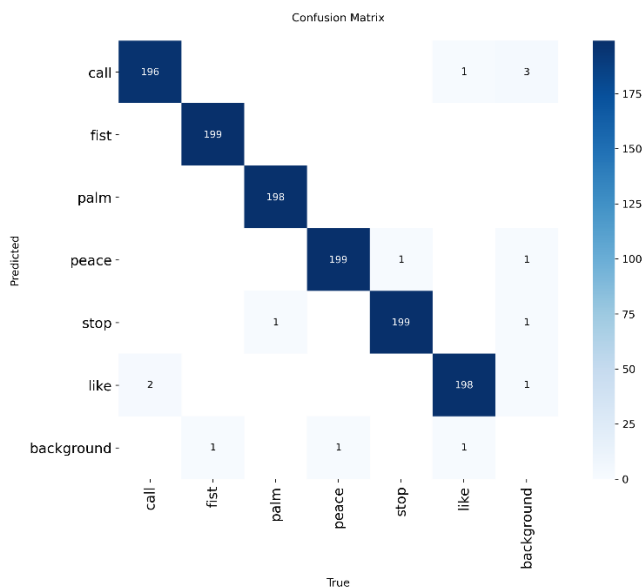
Precision đạt 1.00 tại độ tin cậy ≈ 0.949 . Precision duy trì ở mức rất cao trong toàn bộ khoảng độ tin cậy. Mô hình có rất ít dự đoán sai.



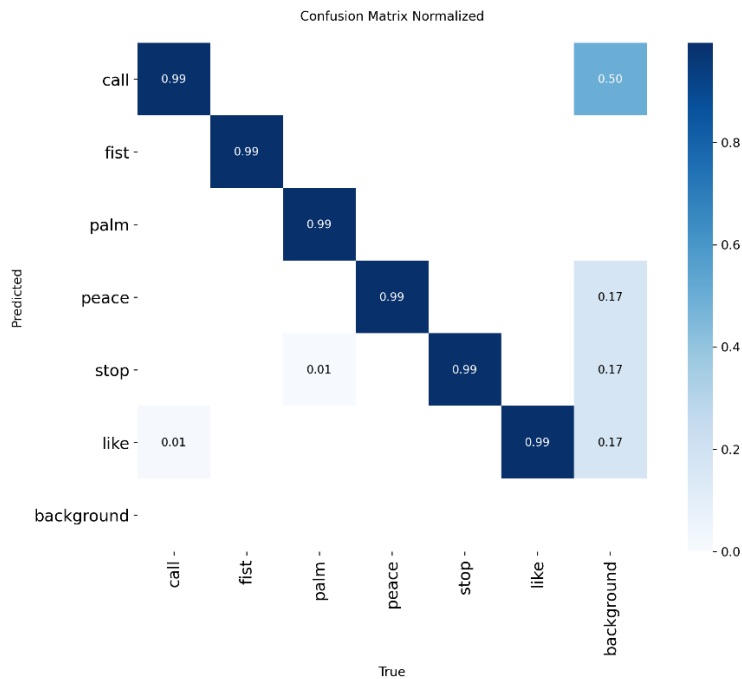
Hình 3.15 Đường cong Recall - Confidence

Recall duy trì gần 1.0 ở vùng độ tin cậy thấp và trung bình và giảm dần khi độ tin cậy tăng cao, phù hợp với lý thuyết: khi mô hình “khắc khe” hơn thì sẽ bỏ sót một số đối tượng.

c. Phân tích confusion matrix



Hình 3.16 Confusion matrix của mô hình



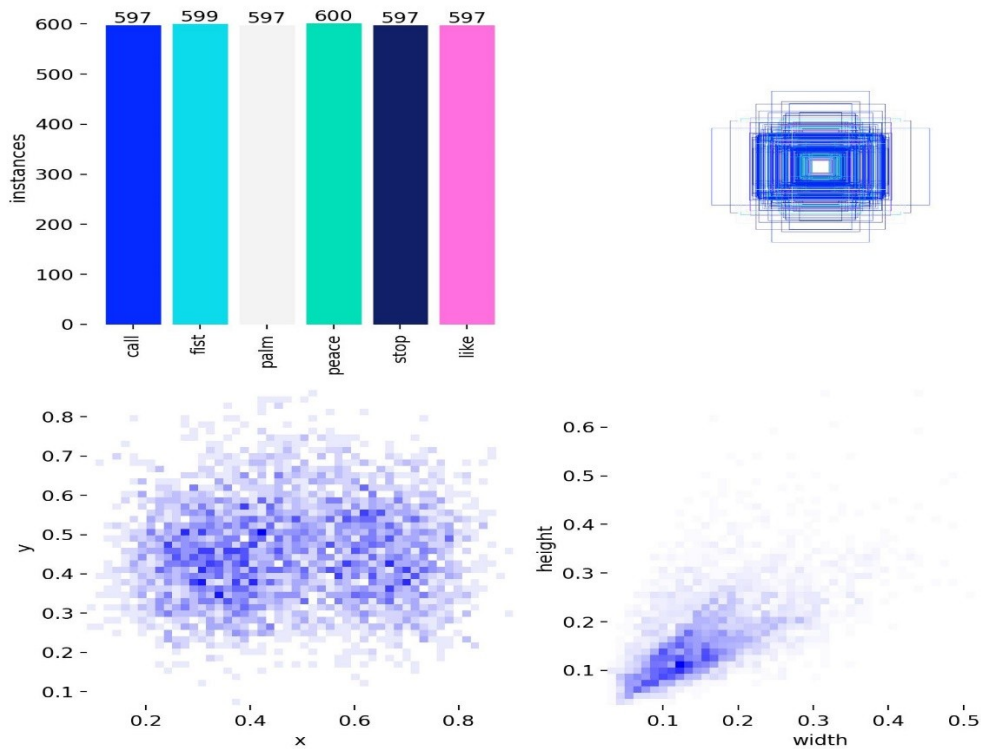
Hình 3.17 Confusion matrix chuẩn hóa

Các giá trị tập trung chủ yếu trên đường chéo chính, cho thấy tỷ lệ dự đoán đúng rất cao. Cụ thể trên tập kiểm thử (Test set) 1.200 ảnh, mô hình đạt kết quả:

- call, fist, peace và stop: Tỷ lệ nhận diện chính xác đạt 0,99.
- palm: Đạt độ chính xác 0,99, có 1% nhầm lẫn sang nền (background).
- like: Đạt độ chính xác 0,99, có 1% nhầm lẫn nhận diện sai.

Nhìn chung, số lượng nhầm lẫn rất thấp và không làm ảnh hưởng đáng kể đến hiệu năng tổng thể của mô hình.

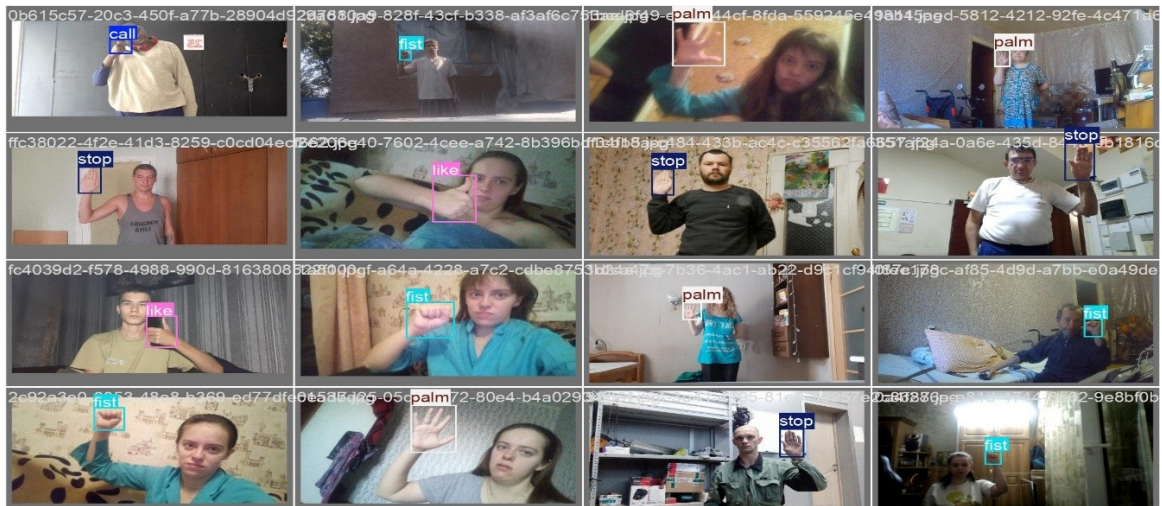
d. Phân tích dữ liệu và phân bố nhãn

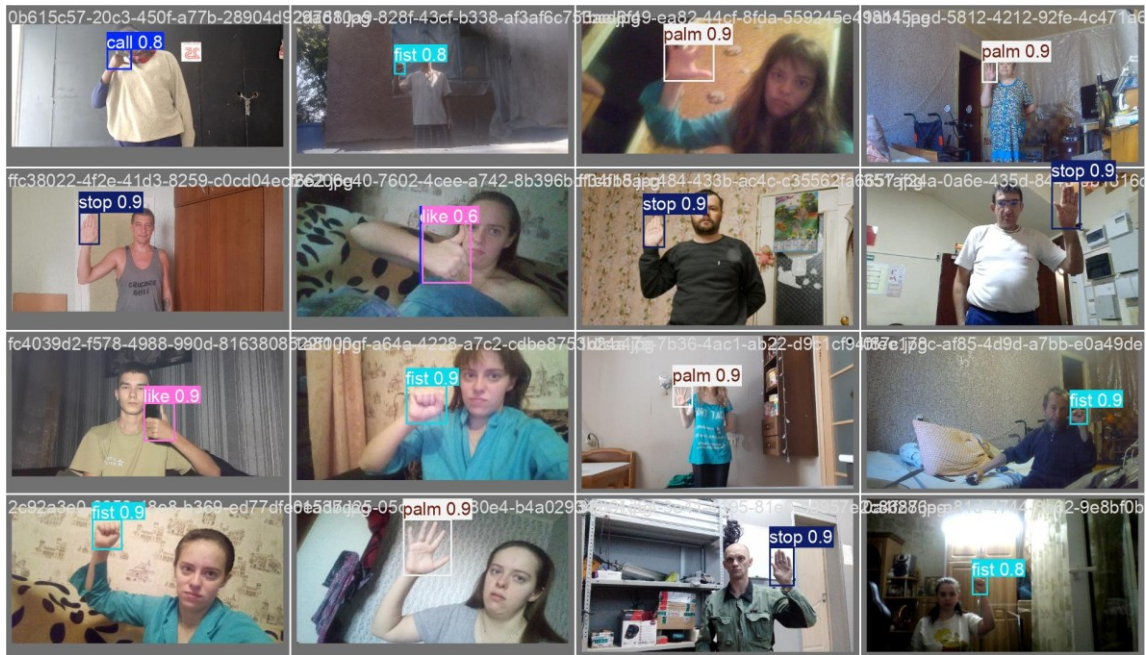


Hình 3.18 Phân bố dữ liệu và hộp giới hạn trong tập huấn luyện

Số lượng mẫu dự đoán giữa các lớp trong tập huấn luyện lần 2 ghi nhận mức độ đồng đều rất cao: call (597), fist (599), palm (597), peace (600), stop (597), like (597). Việc bổ sung dữ liệu đã giúp triệt tiêu sự chênh lệch (đặc biệt ở lớp palm so với lần huấn luyện 1), mang lại tập dữ liệu cân bằng lý tưởng. Hộp giới hạn vẫn tập trung chủ yếu ở vùng trung tâm ảnh, phù hợp với đặc điểm bài toán nhận diện cử chỉ tay trước webcam.

e. Kết quả dự đoán trên tập Validation

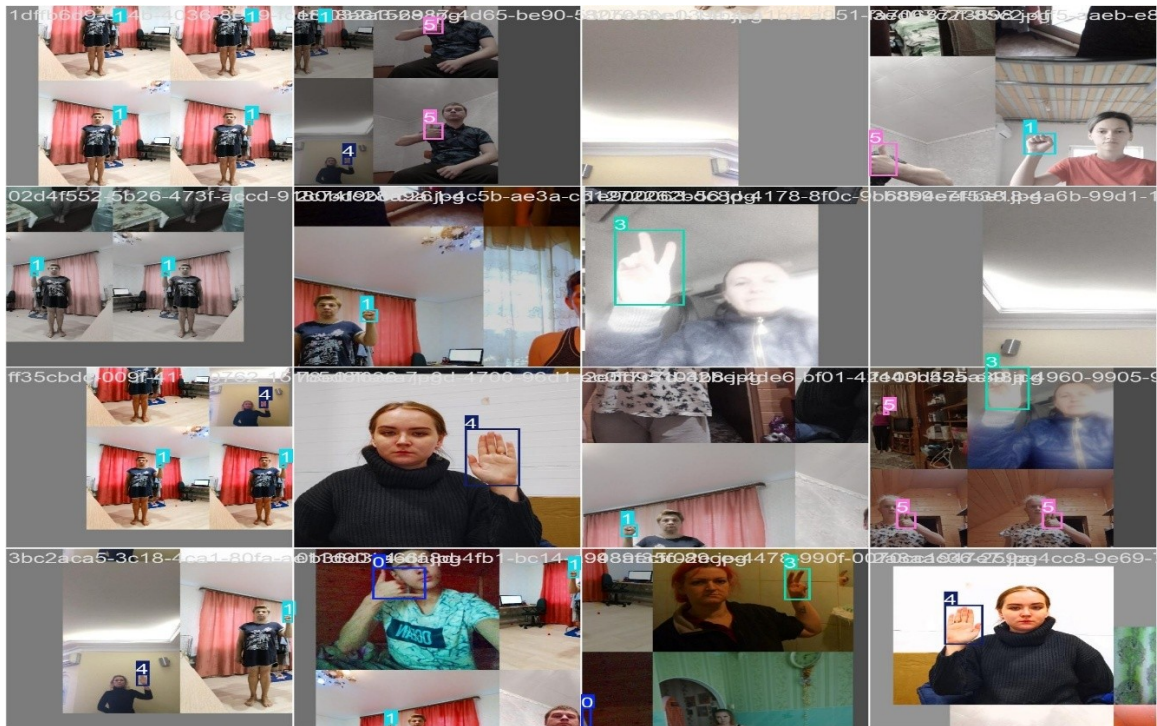


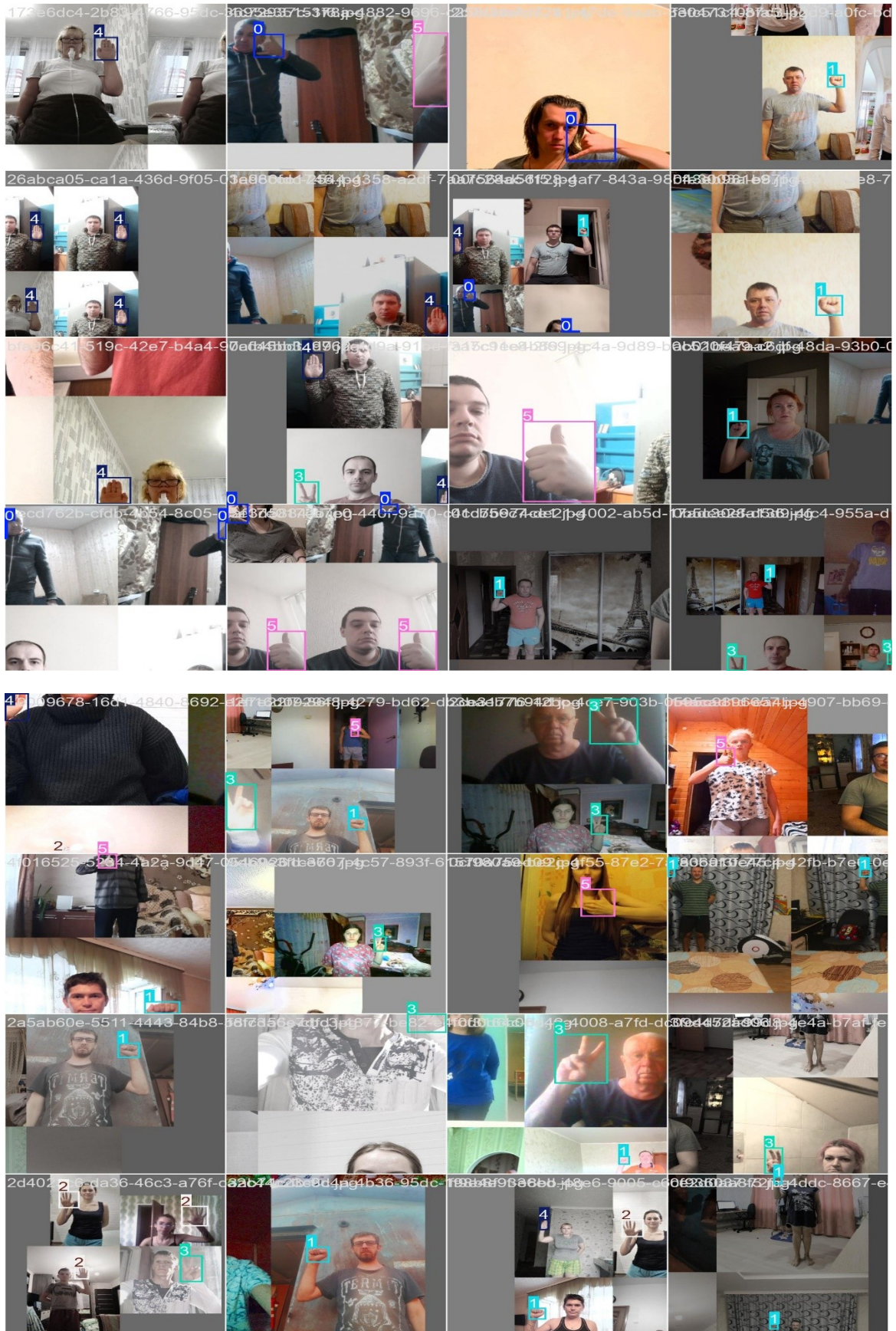


Hình 3.19 So sánh ground truth và kết quả dự đoán

Mô hình phát hiện chính xác vị trí bàn tay. Phân loại đúng các cử chỉ với độ tin cậy cao ($\sim 0.8 - 0.95$). Các hộp giới hạn bám sát đối tượng. Chứng tỏ mô hình có khả năng tổng quát hóa tốt trên dữ liệu thực tế.

f. Minh họa quá trình huấn luyện





Hình 3.20 Một số batch dữ liệu trong quá trình huấn luyện

Dữ liệu huấn luyện đa dạng về: Môi trường, ánh sáng, góc nhìn và đối tượng. Mô hình học được đặc trưng phong phú và tăng khả năng tổng quát hóa.

Nghiên cứu đối chứng hiệu năng mô hình

Để chứng minh tính hiệu quả của chiến lược tinh chỉnh và gia tăng quy mô tập dữ liệu, đề tài sử dụng chính kết quả của lần huấn luyện 1 (khởi tạo từ trọng số COCO, sử dụng 4.800 ảnh) làm cơ sở đối chứng. Hiệu năng của đối chứng này được so sánh trực tiếp với mô hình tối ưu đạt được sau lần huấn luyện 2 (tinh chỉnh từ trọng số tốt nhất của lần 1 và mở rộng dữ liệu lên 6.000 ảnh). Việc mở rộng tập dữ liệu từ 800 ảnh mỗi lớp lên 1.000 ảnh mỗi lớp kết hợp với học chuyển giao đã mang lại những cải thiện rõ rệt:

Về các chỉ số: Mặc dù $mAP@50$ của cả hai lần huấn luyện đều tiệm cận mức tuyệt đối (tăng nhẹ từ 0.994 lên 0.995), chỉ số $mAP@50-95$ (đòi hỏi độ chính xác định vị hộp giới hạn khắt khe hơn) đã có sự cải thiện đáng kể từ ~ 0.830 lên ~ 0.840 .

Về khả năng hạn chế bỏ sót: Đường cong Recall của lần 2 giữ vững được mức cao ngay cả khi ngưỡng độ tin cậy vượt mức 0.85, cho thấy mô hình sau khi tinh chỉnh giảm thiểu đáng kể tỷ lệ bỏ sót đối tượng ở các khung hình khó.

Về ma trận nhầm lẫn: Tác dụng lớn nhất của lần 2 là khắc phục được các lỗi nhầm lẫn cục bộ. Các sai sót khi nhận diện giữa các lớp có hình thái tương đồng (lớp palm và like) khi xuất hiện góc nghiêng hoặc nền phức tạp ở lần 1 đã được cải thiện đáng kể.

Kết quả đối chứng khẳng định chiến lược huấn luyện hai giai đoạn là hoàn toàn hợp lý, cung cấp nền tảng vững chắc để triển khai hệ thống vào môi trường thời gian thực.

Bảng 3.4 So sánh hiệu năng mô hình giữa lần huấn luyện 1 và lần huấn luyện 2

Tiêu chí đánh giá	Phiên huấn luyện 1	Phiên huấn luyện 2	Đánh giá sự cải thiện
Quy mô dữ liệu	4.800 ảnh	6.000 ảnh	Tăng độ đa dạng, bao quát nhiều góc độ và điều kiện ánh sáng hơn.
$mAP@50$	0.994	0.995	Đạt độ chính xác rất cao và duy trì ổn định

mAP@50-95	~0.830	~0.840	Cải thiện khả năng định vị hộp giới hạn khắt khe hơn.
Recall	Giảm nhanh khi ngưỡng độ tin cậy > 0.8	Giữ vững mức cao đến ngưỡng độ tin cậy > 0.85	Mô hình L2 cải tiến hơn, giảm đáng kể tỷ lệ bỏ sót đối tượng ở các khung hình khó.
Nhầm lẫn cục bộ	Tồn tại sai sót ở lớp palm và like	Khắc phục đáng kể	Triệt tiêu các lỗi nhầm lẫn hình thái khi tiếp xúc với các mẫu dữ liệu phức tạp.

3.4 Đánh giá mô hình trên tập test

Mô hình tốt nhất sau lần huấn luyện hai được tinh chỉnh trên tổng quy mô bộ dữ liệu mở rộng 6.000 ảnh. Tập kiểm thử này gồm 1.200 ảnh (tương đương 20% dữ liệu, phân bổ đều 200 ảnh cho mỗi lớp), được bảo toàn và không tham gia vào bất kỳ giai đoạn huấn luyện hay kiểm định trước đó.

Kết quả đánh giá cho thấy mô hình đạt hiệu năng rất cao với mAP@50 xấp xỉ 0.995 và mAP@50-95 dao động trong khoảng 0.83 đến 0.84. Các chỉ số Precision và Recall của hầu hết các lớp đều đạt gần mức 1.0, mô hình ít dự đoán sai và ít bỏ sót đối tượng. Xét từng lớp, các cử chỉ call, fist, peace và stop được nhận diện gần như tuyệt đối với số lượng dự đoán đúng rất cao. Hai lớp palm và like vẫn còn một số nhầm lẫn nhỏ, chủ yếu do sự tương đồng về hình dạng bàn tay trong một số góc nhìn nhất định. Tuy nhiên, tỷ lệ sai này rất thấp và không ảnh hưởng đáng kể đến hiệu năng tổng thể của mô hình.

Nhìn chung, kết quả trên tập test cho thấy mô hình đạt hiệu năng cao trên tập dữ liệu kiểm thử độc lập cùng nguồn dữ liệu và bước đầu cho thấy khả năng tổng quát hóa tốt đối với các mẫu chưa được sử dụng trong huấn luyện. Trong phạm vi nghiên cứu, mô hình cơ bản đáp ứng các yêu cầu đặt ra đối với bài toán nhận diện cử chỉ tay.

3.5 Giao diện hệ thống

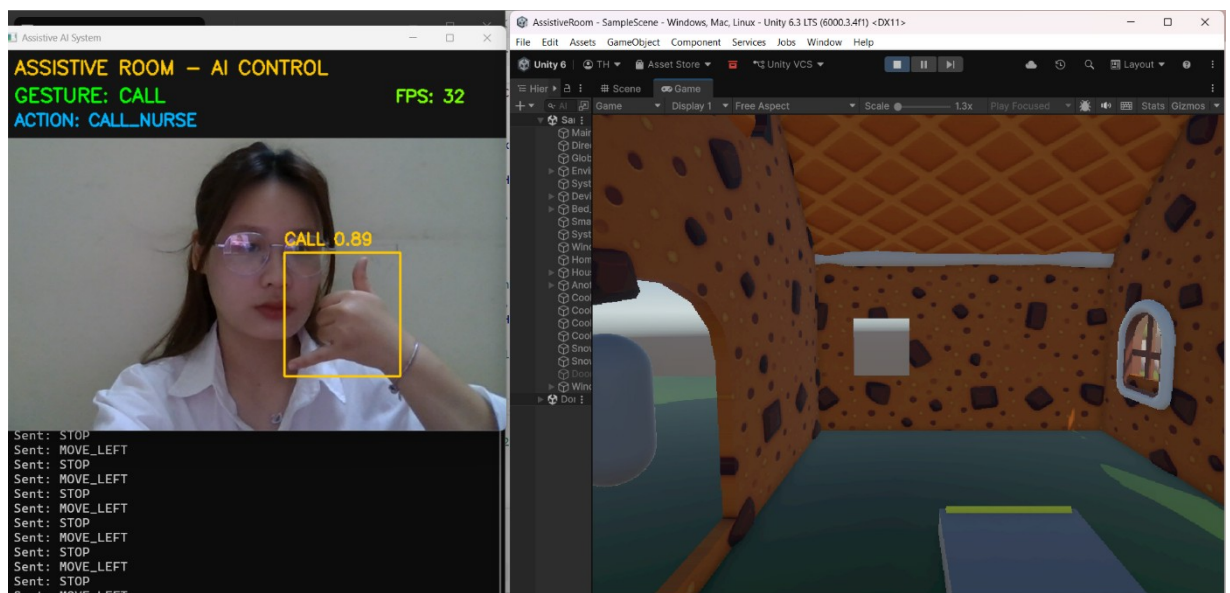
Giao diện màn hình chính



Hình 3.21 Môi trường mô phỏng 3D trong Unity

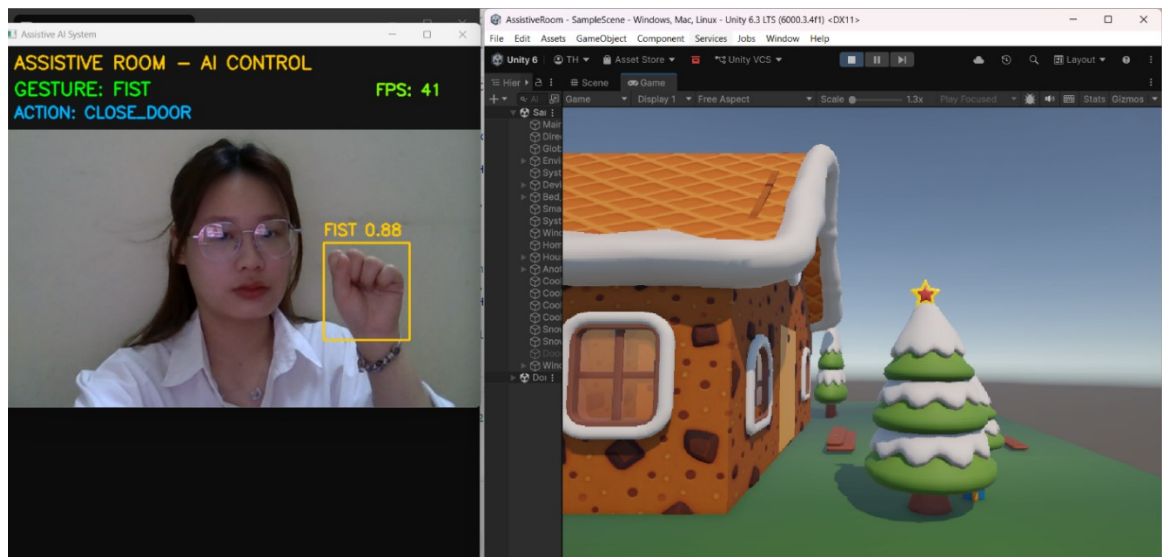
Môi trường mô phỏng 3D được xây dựng trong Unity, bao gồm ngôi nhà với cửa ra vào, các vật thể nội thất và đèn có thể điều khiển. Đây là môi trường ảo nơi các hành động điều khiển được ánh xạ từ cử chỉ tay nhận diện qua webcam sẽ được thực thi và quan sát trực quan.

Kết quả ứng dụng nhận diện cử chỉ tay



Hình 3.22 Cử chỉ call - Kích hoạt hành động CALL_NURSE

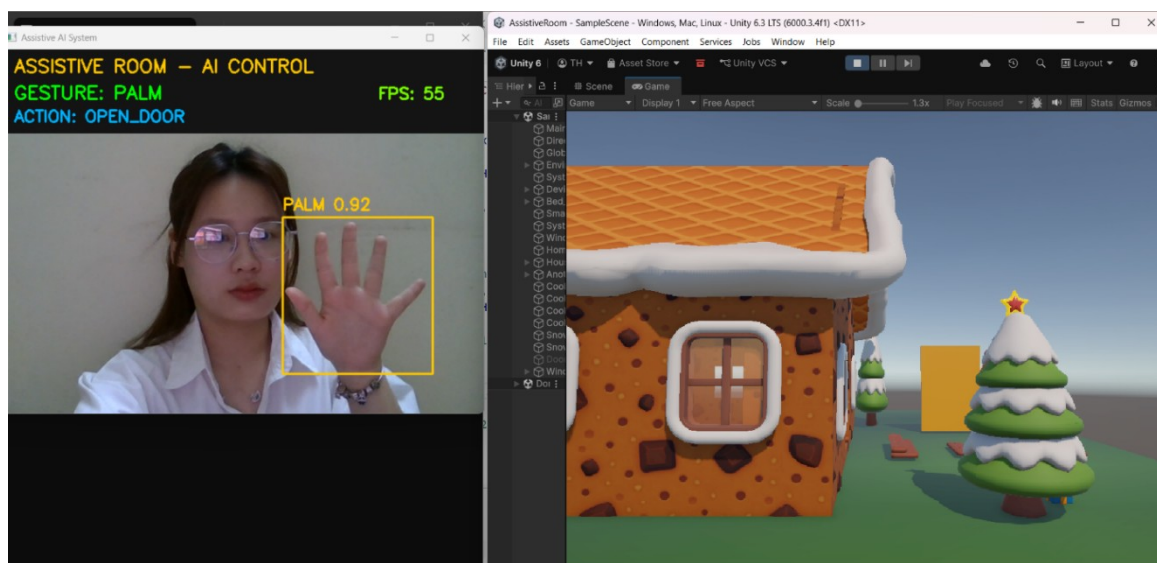
Kết quả nhận diện cử chỉ call (ngón cái và ngón út duỗi) trong điều kiện thời gian thực. Hộp giới hạn màu vàng bám chính xác vùng bàn tay với độ tin cậy 0.89. Hệ thống hiển thị nhãn "GESTURE: CALL" và hành động tương ứng "ACTION: CALL_NURSE" trên giao diện. FPS đạt 32 trên môi trường CPU, đảm bảo phản hồi tức thì. Phía phải màn hình, cảnh Unity phản chiếu trạng thái y tá hình con nhộng xuất hiện sau khi lệnh gọi y tá được kích hoạt.



Hình 3.23 Cử chỉ fist - Kích hoạt hành động CLOSE_DOOR

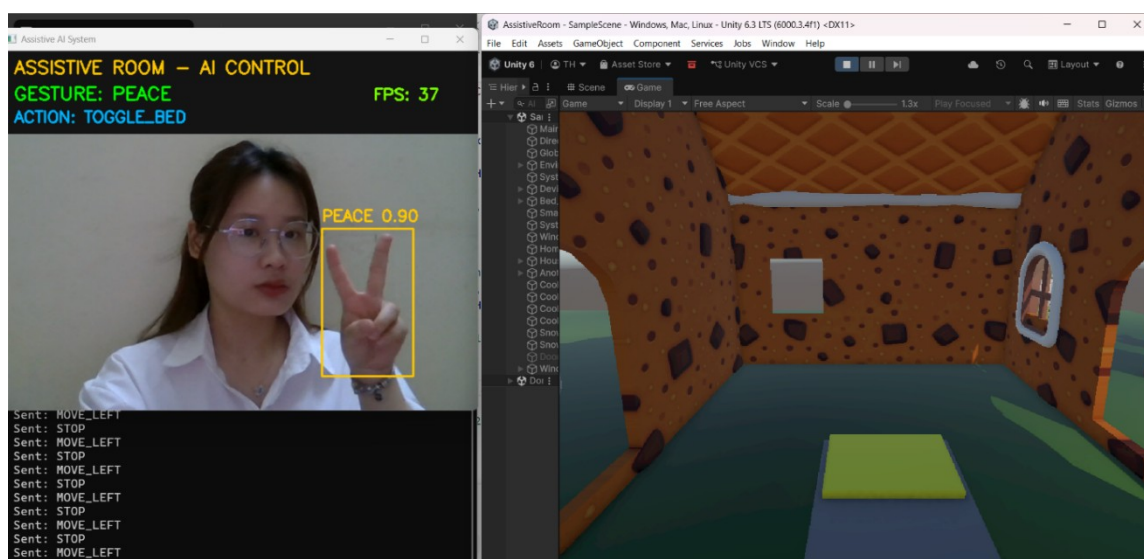
Cử chỉ nắm tay (fist) được nhận diện với độ tin cậy 0.88 và tốc độ xử lý 41 FPS. Hộp giới hạn bám sát vùng bàn tay nắm chặt ở phía bên phải khuôn mặt người

dùng. Hành động "CLOSE_DOOR" được ánh xạ và thực thi tức thì trong môi trường Unity - góc nhìn từ phía ngoài ngôi nhà cho thấy cửa đang ở trạng thái đóng.



Hình 3.24 Cử chỉ palm - Kích hoạt hành động OPEN_DOOR

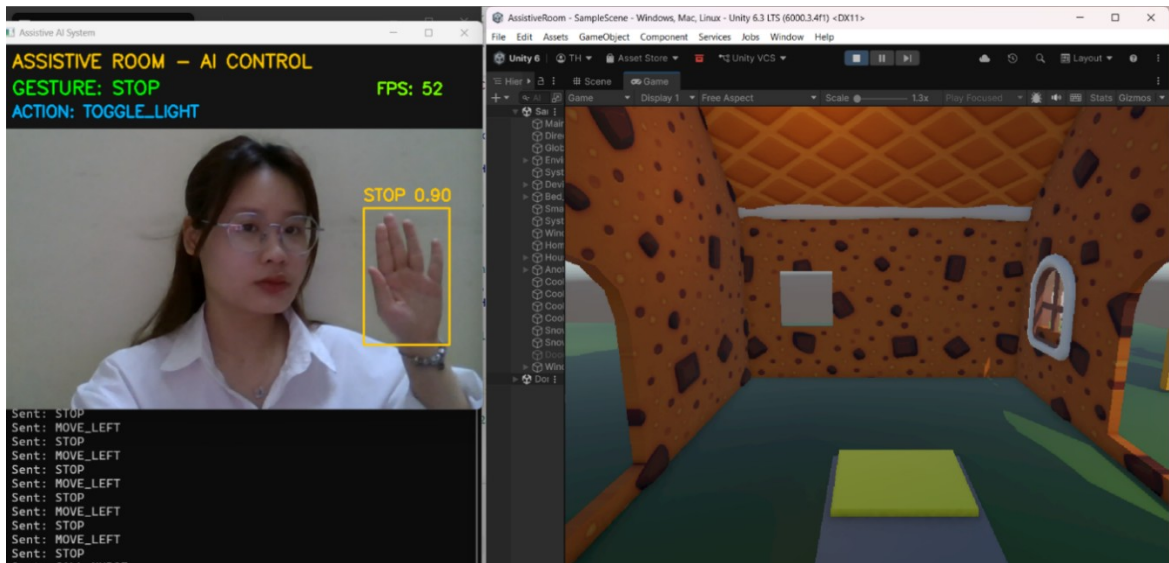
Cử chỉ lòng bàn tay xòe (palm) được nhận diện với độ tin cậy cao nhất trong bộ ảnh demo - 0.92, FPS đạt 55. Hộp giới hạn bao phủ toàn bộ bàn tay xòe rộng với độ chính xác định vị tốt. Hành động "OPEN_DOOR" được kích hoạt, cửa phản hồi ở trạng thái mở - thanh cửa màu vàng dịch chuyển sang phải.



Hình 3.25 Cử chỉ peace - Kích hoạt hành động TOGGLE_BED

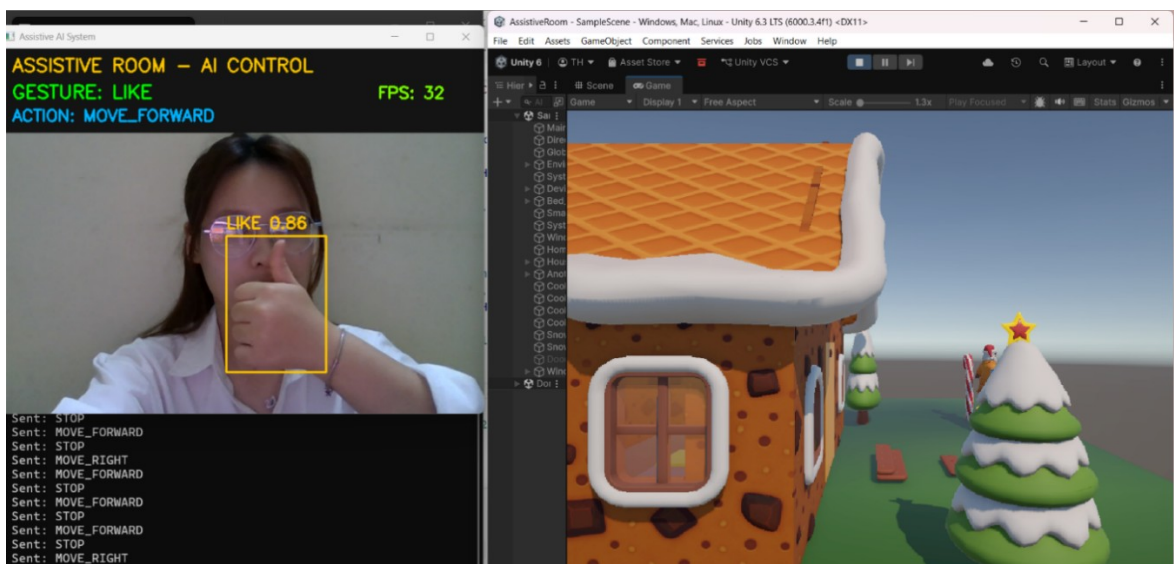
Cử chỉ chữ V (ngón trỏ và ngón giữa duỗi thẳng) được nhận diện với độ tin cậy 0.90 ở tốc độ 37 FPS. Hộp giới hạn định vị chính xác phần bàn tay dưới góc nghiêng nhẹ, cho thấy khả năng xử lý biến thể góc nhìn của mô hình. Hành động

"TOGGLE_BED" - chiếc giường (đệm màu xanh lá) chuyển sang trạng thái mới, mô phỏng thao tác nâng giường bệnh viện.



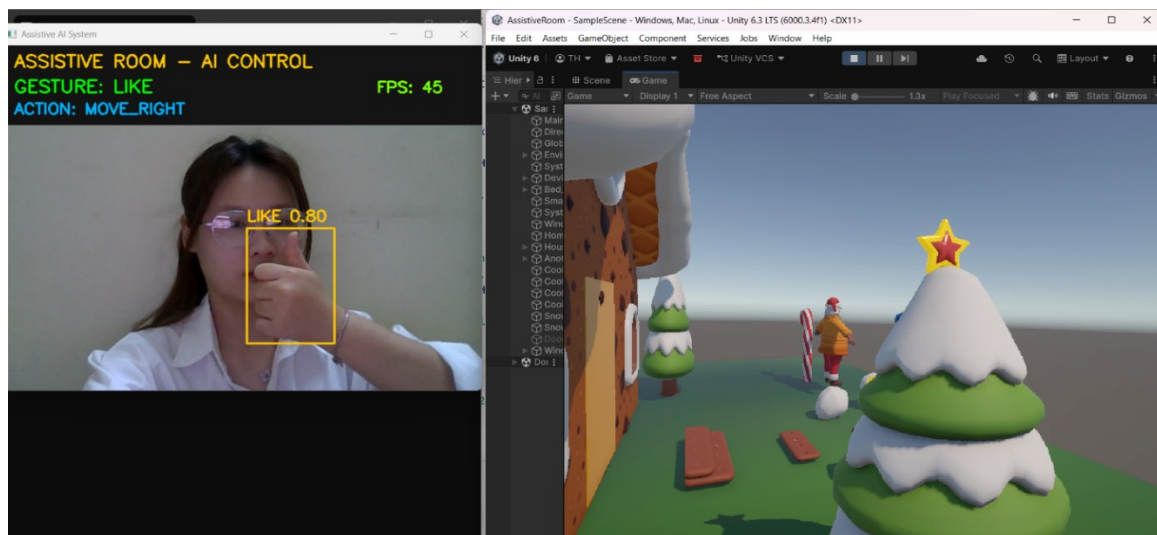
Hình 3.26 Cử chỉ stop - Kích hoạt hành động TOGGLE_LIGHT

Cử chỉ dừng (bàn tay giơ thẳng, ngón cái gấp vào) được nhận diện với độ tin cậy 0.90 và FPS 52. Hộp giới hạn bao phủ toàn bộ lòng bàn tay hướng về phía camera. Hành động "TOGGLE_LIGHT" được kích hoạt tức thì, đèn trong phòng chuyển từ trạng thái tắt sang bật sáng trong môi trường Unity. Cử chỉ stop có cấu hình bàn tay đơn giản, rõ ràng, giúp mô hình phân biệt chính xác so với các cử chỉ khác, phù hợp cho thao tác bật/tắt thiết bị điện một chạm.



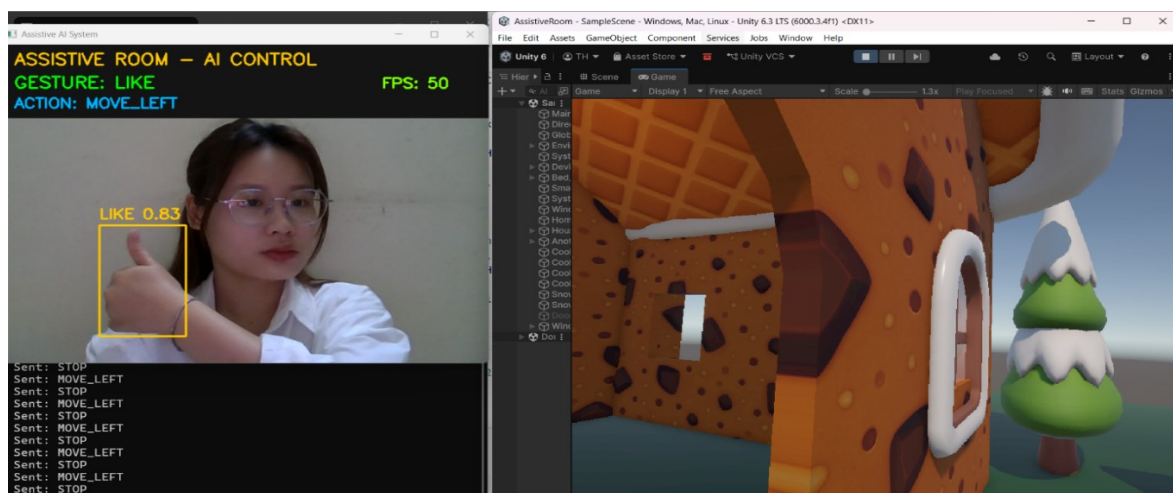
Hình 3.27 Cử chỉ like - Hành động MOVE_FORWARD

Cử chỉ ngón cái giơ lên (like) khi người dùng giơ tay thẳng, hệ thống ánh xạ thành hành động "MOVE_FORWARD" với độ tin cậy 0.86 và FPS 32. Khi tâm hộp giới hạn rơi vào vùng trung tâm (1/3 giữa khung hình), hệ thống ánh xạ thành hành động "MOVE_FORWARD", nhân vật di chuyển tiến thẳng trong môi trường mô phỏng Unity. Đây là điểm đặc trưng của cơ chế ánh xạ vị trí không gian từ một cử chỉ duy nhất thành ba lệnh điều hướng khác nhau, không yêu cầu huấn luyện lại mô hình.



Hình 3.28 Cử chỉ like - Hành động MOVE_RIGHT

Cử chỉ ngón cái giơ lên (like) khi người dùng nghiêng tay về phía phải sao cho tâm hộp giới hạn rơi vào 1/3 bên phải của khung hình, hệ thống ánh xạ thành hành động "MOVE_RIGHT" với độ tin cậy 0.80 và FPS 45. Độ tin cậy ở mức 0.80 phản ánh sự thay đổi về hình dạng biểu kiến của bàn tay khi nghiêng góc, tuy nhiên vẫn nằm trên ngưỡng chấp nhận (0.5), đảm bảo lệnh di chuyển sang phải được thực thi chính xác và liên tục trong môi trường Unity.



Hình 3.29 Cử chỉ like - Hành động MOVE_LEFT

Cử chỉ ngón cái giờ lên (like) khi người dùng nghiêng tay về phía trái sao cho tâm hộp giới hạn rơi vào 1/3 bên trái khung hình, hệ thống ánh xạ thành hành động "MOVE_LEFT" với độ tin cậy 0.83 và FPS 50. Đây là kết quả minh chứng cho tính linh hoạt của cơ chế điều hướng dựa trên vị trí không gian: chỉ với một cử chỉ like duy nhất, hệ thống có thể phân biệt và thực thi ba lệnh điều hướng độc lập (MOVE_FORWARD, MOVE_RIGHT, MOVE_LEFT) mà không cần mở rộng tập nhãn hay huấn luyện thêm lớp dữ liệu mới, giảm đáng kể chi phí tính toán và thời gian.

3.6 Đánh giá hiệu năng thời gian thực và độ bền vững của hệ thống

Để kết quả mang tính khách quan, thay vì chỉ ghi nhận tốc độ khung hình tại một thời điểm bất kỳ, thực nghiệm được thiết kế để hệ thống chạy liên tục trong 5 phút cho mỗi bối cảnh môi trường khác nhau. Toàn bộ quá trình đo lường được thực hiện trên CPU Intel Core i5-12500H, RAM 8GB mà không có sự can thiệp của GPU. Đồng thời, luồng video đầu vào từ webcam được tiền xử lý và thu nhỏ kích thước về 320x180 pixel nhằm tối ưu hóa chi phí tính toán.

Bảng 3.5 Thống kê hiệu năng trong các điều kiện môi trường

Điều kiện môi trường	FPS	Độ trễ	Tỷ lệ nhận diện đúng
Ánh sáng phòng chuẩn, khoảng cách 0.5m	~45 FPS	< 50ms	98%
Thiếu sáng nhẹ, khoảng cách 1m	~35 FPS	< 70ms	92%
Góc nghiêng bàn tay 45 độ	~40 FPS	< 60ms	85%
Nền phức tạp	~38 FPS	< 60ms	88%

Kết luận: Mô hình thể hiện tính bền vững cao, không xảy ra hiện tượng treo hay sụt giảm hiệu năng nghiêm trọng khi môi trường thay đổi, đáp ứng trọn vẹn các yêu cầu phi chức năng đã đề ra.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Kết quả đạt được

Qua quá trình thực hiện khóa luận tốt nghiệp, em đã nghiên cứu, xây dựng được mô hình nhận diện cử chỉ tay thời gian thực bằng YOLO và ứng dụng trong điều khiển thao tác. Các mục tiêu đặt ra ban đầu về cơ bản đã hoàn thành cụ thể là:

Em đã thu thập và sử dụng thành thạo công cụ LabelImg để gán nhãn thủ công cho bộ dữ liệu lớn gồm 6.000 ảnh thuộc 6 lớp cử chỉ tay (call, fist, like, palm, peace, stop) theo đúng định dạng YOLO. Quá trình này giúp em hiểu rõ về cấu trúc dữ liệu đầu vào cần thiết cho mạng học sâu.

Em đã triển khai môi trường huấn luyện trên nền tảng Google Colab, khai thác hiệu quả sức mạnh tính toán của GPU Tesla T4. Kết hợp ngôn ngữ Python, framework Ultralytics và thư viện PyTorch, em đã huấn luyện thành công kiến trúc mạng YOLOv12m trên tổng số 6.000 ảnh dữ liệu qua hai lần huấn luyện (4.800 ảnh lên đến 6.000 ảnh). Áp dụng chiến lược tinh chỉnh từ trọng số COCO, mô hình hội tụ nhanh và đạt hiệu năng rất cao. Khi được đánh giá độc lập trên tập kiểm thử (gồm 1.200 ảnh trích xuất từ bộ 6.000 ảnh gốc), mô hình YOLOv12 sau tinh chỉnh đạt kết quả cao với mAP@50 đạt 0.995 và mAP@50-95 đạt 0.840, Precision và Recall đều xấp xỉ 1,0. Kết quả này đáp ứng hoàn toàn các tiêu chí đánh giá ban đầu đặt ra cho hệ thống.

Không dừng lại ở việc tạo ra mô hình lý thuyết, em đã xây dựng thành công một ứng dụng thao tác điều khiển sử dụng giao diện đồ họa bằng PyQt6, kết hợp với thư viện OpenCV để xử lý luồng video trực tiếp. Khi triển khai thời gian thực trên môi trường CPU (Intel Core i5-12500H), hệ thống duy trì tốc độ khung hình trung bình từ 38 đến 51 FPS, độ trễ dưới 50ms. Điều này đảm bảo khả năng phản hồi tức thì các lệnh điều khiển như bật/tắt đèn, đóng/mở cửa và di chuyển vị trí (tiền, trái, phải) trong môi trường mô phỏng 3D Unity. Hệ thống mô phỏng đã cho thấy khả năng hỗ trợ tương tác không chạm trong một số kịch bản điều khiển cơ bản. Kết quả bước đầu cho thấy tiềm năng ứng dụng của hệ thống trong các không gian sống thông minh, góp phần hỗ trợ người hạn chế vận động thao tác với các thiết bị xung quanh thuận tiện hơn.

Hạn chế

Bên cạnh những kết quả đạt được, hệ thống vẫn còn một số hạn chế cần nhìn nhận thẳng thắn:

Bộ dữ liệu huấn luyện chỉ sử dụng 6.000 ảnh (khoảng 1% bộ HaGRID gốc), khiến khả năng tổng quát hóa trong các điều kiện ánh sáng cực đoan hoặc nền rất phức tạp còn hạn chế.

Hệ thống mới chỉ tập trung nhận diện được các cử chỉ tay tĩnh, chưa hỗ trợ nhận diện các cử chỉ động có sự thay đổi theo quỹ đạo thời gian (như thao tác vẩy tay, vuốt).

Ứng dụng chưa xử lý logic điều khiển khi hai tay thực hiện hai cử chỉ khác nhau đồng thời trong cùng một khung.

Số lượng cử chỉ được hỗ trợ còn giới hạn ở sáu lớp, chưa mở rộng sang các cử chỉ phức tạp hơn phục vụ ngôn ngữ ký hiệu hoặc điều khiển đa lệnh.

Hướng phát triển trong tương lai

Dựa trên những kết quả và hạn chế đã xác định, một số hướng phát triển tiếp theo được đề xuất như sau:

Mở rộng bộ dữ liệu và số lớp cử chỉ: Tăng số lượng ảnh huấn luyện lên toàn bộ 18 lớp HaGRID hoặc thu thập thêm dữ liệu thực tế nâng cao khả năng tổng quát hóa và mở rộng tập lệnh điều khiển.

Tích hợp nhận diện đa tay: Bổ sung logic xử lý khi hai tay thực hiện cử chỉ đồng thời, cho phép kết hợp các lệnh phức tạp hơn trong môi trường điều khiển thực tế.

Cải thiện giao diện thân thiện hơn, bổ sung chế độ hiệu chỉnh ngưỡng độ tin cậy trực tiếp trên ứng dụng và hỗ trợ lưu nhật ký lịch sử lệnh điều khiển phục vụ phân tích sau sử dụng.

TÀI LIỆU THAM KHẢO

- [1] Y. Tian, Q. Ye, and D. Doermann, ‘YOLOv12: Attention-Centric Real-Time Object Detectors’, *ArXiv Prepr. ArXiv250212524*, 2025.
- [2] S. S. Rautaray and A. Agrawal, ‘Vision based hand gesture recognition for human computer interaction: a survey’, *Artif. Intell. Rev.*, vol. 43, no. 1, pp. 1 - 54, 2015, doi: 10.1007/s10462-012-9356-9.
- [3] World Health Organization, ‘World report on hearing’, 2021. [Online]. Available: <https://www.who.int/publications/i/item/world-report-on-hearing>
- [4] P. Molchanov et al., ‘Hand Gesture Recognition with 3D Convolutional Neural Networks’, in *Proc. IEEE Conf. Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2015, pp. 1 - 9.
- [5] A. Krizhevsky, I. Sutskever, and G. E. Hinton, ‘ImageNet Classification with Deep Convolutional Neural Networks’, in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012, vol. 25.
- [6] Abirami Vina, ‘Độ chính xác trung bình (mAP) trong phát hiện đối tượng’. May 28, 2025. [Online]. Available: <https://www.ultralytics.com/vi/blog/mean-average-precision-map-in-object-detection>.
- [7] A. Kapitanov, K. Kvanchiani, A. Nagaev, R. Kraynov, and A. Makhliarchuk, ‘HaGRID - HAnd Gesture Recognition Image Dataset’, in *Proceedings of IEEE/CVF WACV 2024*, 2024, pp. 4572 - 4581.
- [8] roboflow, ‘YOLO’. [Online]. Available: <https://roboflow.com/formats/yolo>
- [9] GPU Tesla T4, *Google Colab*. [Online]. Available: <https://colab.research.google.com/>
- [10] *Ultralytics YOLO Python API*. [Online]. Available: <https://docs.ultralytics.com/>
- [11] opencv-software, *OpenCV*. [Online]. Available: <https://docs.opencv.org/>
- [12] ‘Qt for Python’. [Online]. Available: <https://doc.qt.io/qtforpython-6/>